Footland
[Online Footwear Store]

Mini Project Report

Submitted by

Treesa Thomas

Reg. No.: AJC23MCA-2063

In Partial fulfillment for the Award of the Degree of

MASTER OF COMPUTER APPLICATIONS (MCA)

AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS

KANJIRAPPALLY

[Approved by AICTE, Accredited by NAAC, Accredited by NBA.
Koovappally, Kanjirappally, Kottayam, Kerala – 686518]

2024-2025

DEPARTMENT OF COMPUTER APPLICATIONS

AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS

KANJIRAPPALLY

CERTIFICATE

This is to certify that the Project report, "**Footland"** is the bonafide work of **TREESA THOMAS (Regno: AJC23MCA-2063)** in partial fulfillment of the requirements for the award of the Degree of Master of Computer Applications under **Amal Jyothi College of Engineering Autonomous, Kanjirappally** during the year 2024-25.

Guide Name
Jetty Benjamin

Coordinator Name
Binumon Joseph

Rev. Fr. Dr. Rubin Thottupurathu Jose
Head of the Department

DECLARATION

I hereby declare that the project report **"Footland"** is a bonafide work done at Amal Jyothi College of Engineering, towards the partial fulfilment of the requirements for the award of the Master of Computer Applications (MCA) from Amal Jyothi College of Engineering Autonomous during the academic year 2024-2025.

Date: **TREESA THOMAS**
KANJIRAPPALLY **Reg: AJC23MCA-2063**

ACKNOWLEDGEMENT

First and foremost, I thank God almighty for his eternal love and protection throughout the project. I take this opportunity to express my gratitude to all who helped me in completing this project successfully. It has been said that gratitude is the memory of the heart. I wish to express my sincere gratitude to our Director (Administration) **Rev. Fr. Dr. Roy Abraham Pazhayaparampil** and Principal **Dr. Lillykutty Jacob** for providing good faculty for guidance.

I owe a great depth of gratitude towards our Head of the Department **Rev. Fr. Dr. Rubin Thottupurathu Jose** for helping us. I extend my whole hearted thanks to the project coordinator **Binumon Joseph** for his valuable suggestions and for overwhelming concern and guidance from the beginning to the end of the project. I would also express sincere gratitude to my guide **Jetty Benjamin** for her inspiration and helping hand.

I thank our beloved teachers for their cooperation and suggestions that helped me throughout the project. I express my thanks to all my friends and classmates for their interest, dedication, and encouragement shown towards the project. I convey my hearty thanks to my family for the moral support, suggestions, and encouragement to make this venture a success.

TREESA THOMAS

ABSTRACT

.

Footland is an e-commerce platform designed for purchasing footwear from any brand, offering a seamless, engaging online shopping experience. The website caters to guest users, registered customers, and admin users, with plans for vendor management tools in future updates. Guest users can explore products, view promotions, and add items to their cart without logging in. Registered users can access guest features plus manage personal profiles, track orders, save wish lists, and manage items for future purchases. Admin users oversee the site through a robust dashboard, handling product listings, customer orders, user accounts, and content management. Advanced features like analytics offer insights into sales performance, including identifying best-selling products.

Footland aims to provide intuitive product browsing, secure checkouts, personalized recommendations, and responsive design, driving customer satisfaction, loyalty, and sales growth. This platform is set to transform online footwear shopping by delivering an enriched user experience with a strong focus on functionality and security.

CONTENT

SL. NO	TOPIC	PAGE NO
1	INTRODUCTION	2
1.1	PROJECT OVERVIEW	3
1.2	PROJECT SPECIFICATION	3
2	SYSTEM STUDY	5
2.1	INTRODUCTION	6
2.2	EXISTING SYSTEM	6
2.3	DRAWBACKS OF EXISTING SYSTEM	7
2.4	PROPOSED SYSTEM	7
2.5	ADVANTAGES OF PROPOSED SYSTEM	8
3	REQUIREMENT ANALYSIS	9
3.1	FEASIBILITY STUDY	10
3.1.1	ECONOMICAL FEASIBILITY	10
3.1.2	TECHNICAL FEASIBILITY	11
3.1.3	BEHAVIORAL FEASIBILITY	11
3.1.4	FEASIBILITY STUDY QUESTIONNAIRE	11
3.2	SYSTEM SPECIFICATION	13
3.2.1	HARDWARE SPECIFICATION	13
3.2.2	SOFTWARE SPECIFICATION	13
3.3	SOFTWARE DESCRIPTION	13
3.3.1	DJANGO	13
3.3.2	MYSQL	14
4	SYSTEM DESIGN	15
4.1	INTRODUCTION	16
4.2	UML DIAGRAM	16
4.2.1	USE CASE DIAGRAM	17
4.2.2	SEQUENCE DIAGRAM	19
4.2.3	STATE CHART DIAGRAM	21
4.2.4	ACTIVITY DIAGRAM	23
4.2.5	CLASS DIAGRAM	25
4.2.6	OBJECT DIAGRAM	27
4.2.7	COMPONENT DIAGRAM	28

4.2.8	DEPLOYMENT DIAGRAM	29
4.3	USER INTERFACE DESIGN USING FIGMA	30
4.4	DATABASE DESIGN	33
5	SYSTEM TESTING	39
5.1	INTRODUCTION	40
5.2	TEST PLAN	40
5.2.1	UNIT TESTING	41
5.2.2	INTEGRATION TESTING	41
5.2.3	VALIDATION TESTING	42
5.2.4	USER ACCEPTANCE TESTING	42
5.2.5	AUTOMATION TESTING	42
5.2.6	SELENIUM TESTING	43
6	IMPLEMENTATION	53
6.1	INTRODUCTION	54
6.2	IMPLEMENTATION PROCEDURE	54
6.2.1	USER TRAINING	54
6.2.2	TRAINING ON APPLICATION SOFTWARE	55
6.2.3	SYSTEM MAINTENANCE	55
7	CONCLUSION & FUTURE SCOPE	58
7.1	CONCLUSION	59
7.2	FUTURE SCOPE	59
8	BIBLIOGRAPHY	61
9	APPENDIX	63
9.1	SAMPLE CODE	64
9.2	SCREEN SHOTS	91

List of Abbreviations

- UML - Unified Modelling Language
- ORM - Object-Relational Mapping
- MVT - Model-View-Template
- MVC - Model-View-Controller
- RDBMS - Relational Database Management System
- 1NF - First Normal Form
- 2NF - Second Normal Form
- 3NF - Third Normal Form
- IDE - Integrated Development Environment
- HTML - HyperText Markup Language
- JS - JavaScript
- CSS - Cascading Style Sheets
- AJAX - Asynchronous JavaScript and XML
- JSON - JavaScript Object Notation
- API - Application Programming Interface
- UI - User Interface
- HTTP - Hypertext Transfer Protocol
- URL - Uniform Resource Locator
- PK - Primary Key
- FK - Foreign Key
- SQL - Structured Query Language
- CRUD - Create, Read, Update, Delete

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW

The project Footland (online footwear system) is a shopping site for purchasing footwear's.This website aims to purchase foot wears from any brand at any time. The project conatins guest users, registered users, admin users. The guest user can go through the site without login , in which they can check the product add to cart. The registered users can login with their mail id and password. They can visit the site , manage or update the profile, view the offers, and purchase and track it.

The admin users can manage product listings, including adding, editing, and removing products oversee customer accounts. View the product analytics.

The development of a footwear e-commerce website aims to create a seamless and engaging online shopping experience for consumers, offering a wide range of footwear products from various brands and categories. Key functionality will encompass intuitive product browsing, advanced search and filtering options, secure checkout processes, and comprehensive user account management.

Admin users will benefit from a robust dashboard to manage products, orders, promotions, and customer interactions, while vendor users will have tools to manage their inventory and sales. By integrating responsive design, personalized recommendations, and top-notch security features, the website aspires to enhance user satisfaction and streamline operations, ultimately driving sales growth and fostering customer loyalty. This abstract outlines the primary goals and functionality of the footwear e-commerce platform, emphasizing its potential to transform the online footwear shopping landscape.

PROJECT SPECIFICATION

1. Guest Users

- Browse Products: View product categories and individual product pages.
- Search: Use a search bar to find specific products.
- View Promotions: Access current sales, discounts, and special offers.
- Add to Cart: Select products and add them to the shopping cart.

2. Registered Users (Customers)

- All Guest User Functionality: Access all features available to guest users.
- Account Management: Create, view, and edit personal profiles, including shipping addresses and payment methods.
- Order History: View past orders and their status.
- Wish list Management: Save and manage a wish list for future purchases.

- Add to cart: Manage a cart item for purchasing
- Saved for later:Save and mange for later purchase.

3. Admin Users

- Dashboard: Access an overview of website performance, sales, and user activity.
- Product Management: Add, edit, or remove product listings, including images, descriptions, prices, and inventory levels.
- Order Management: View and manage customer orders.
- Customer Management: View and manage customer accounts, assist with account issues, and communicate with customers.
- Content Management: Update and manage website content, including banners,blog posts, and static pages.
- Settings: Configure website settings, and security features.
- Analytics: view the product sale and knows which product is best selling

CHAPTER 2

SYSTEM STUDY

2.1 INTRODUCTION

A critical stage in the creation of any system is system analysis. Its main objective is to collect and examine data in order to identify issues and provide solutions. The key to this phase is effective communication between system users and developers. In fact, a system analysis should always bethe first step in every system development project.

Here, the system analyst assumes the role of an investigator, carefully evaluating the effectivenessof the current system. This requires determining the system's inputs and outputs as well as the relationship between its activities and the outcomes of the organization.

Information is gathered through a variety of methods, including surveys and interviews. The broad objectives include learning how the system works, identifying problem areas, and suggesting solutions to deal with the problems facing the company.

2.2 EXISTING SYSTEM

2.2.1 NATURAL SYSTEM STUDIED

The Footland e-commerce platform represents a complex and dynamic online retail ecosystem specifically tailored for footwear shopping. This system mirrors the functionality of a natural ecosystem, where different entities interact with each other to create a balanced, functional environment. In this case, the entities include guest users, registered users, admin users each with their own roles and interactions within the platform.

Guest users navigate through the system without the need to log in, allowing them to explore a wide variety of products, check details, and even add items to their cart. This initial interaction mimics the behavior of visitors in a physical store, browsing products without making immediate commitments. Registered users, on the other hand, have more sophisticated interactions with the platform, such as managing their profiles,,cart,wishlist management. These users contribute to the system by providing purchase data and feedback, helping to enhance product offerings and improve customer service.

In parallel, admin users play the role of ecosystem managers, ensuring the platform runs smoothly by overseeing product listings, customer accounts, and promotional activities. They are responsible for curating the online catalog, managing stock levels, and monitoring the performance of sales and customer interactions. Admins also ensure the integrity of the system by removing or updating products, adjusting prices, and addressing customer issues.

2.2.2 DESIGNED SYSTEM STUDIED

The designed system for Footland is a seamless and user-friendly footwear e-commerce platform catering to guest users, registered users, admin users. It offers intuitive product browsing, advanced search and filtering, and a secure checkout. Guest users can explore and add items to the cart without logging in, while registered users manage profiles, view offers, and track orders.

Admin users have access to a comprehensive dashboard to manage products, promotions, and customer accounts, along with sales analytics for informed decision-making. The platform prioritizes security, responsive design, and efficient backend operations, ensuring a smooth experience across devices. The system is built to support future growth, enhancing user satisfaction and driving sales.

2.3 DRAWBACKS OF EXISTING SYSTEM

1. **Limited search and filter options**: Finding specific footwear based on style, occasion, or custom needs can be difficult due to limited filtering options and basic categorization.
2. **Inconsistent product descriptions**: Variations in product listings, such as incomplete details or unclear sizing information, can lead to confusion and dissatisfaction for users.
3. **Lack of advanced personalization**: Most systems fail to provide tailored product recommendations based on user behavior, preferences, foot dimensions, or past purchases.
4. **Insufficient virtual fitting features**: Many platforms do not offer detailed size guides or virtual try-on options, leading to uncertainty in product sizing and fit.
5. **Ads or promotional clutter**: Some platforms rely heavily on ads, promotions, or pop-ups that disrupt the user experience and make navigation less enjoyable.

2.4 PROPOSED SYSTEM

The proposed system is an advanced footwear e-commerce platform that aims to enhance user experience through a comprehensive set of features, catering to guest users, registered customers, admins The system integrates a seamless shopping experience with tailored product recommendations, advanced product search, and enhanced interactivity.Key functionalities include user authentication, personalized product management, detailed product descriptions, and a secure checkout process. Additionally, the platform fosters a vibrant shopping community by encouraging user reviews, feedback, and social interaction. The system is designed to drive customer loyalty, enhance operational efficiency for vendors and admins, and provide a dynamic and interactive shopping environment.

2.5 ADVANTAGES OF PROPOSED SYSTEM

1. **Advanced Product Search and Filters**: The platform offers intelligent filtering options for users to quickly find products based on specific needs, such as size, style, occasion, brand, or price range. This feature improves navigation and makes product discovery more efficient.
2. **Virtual Try-On and Sizing Tools:** Using machine learning, the platform allows users to virtually try on footwear and get accurate size recommendations, enhancing confidence in purchasing decisions and reducing return rates.
3. **Personalized Product Recommendations**: The system provides tailored product recommendations based on user behavior, shopping history, and preferences. This ensures users see relevant products that match their style and needs.
4. **Customizable Footwear**: Users can personalize select footwear items by choosing different designs, materials, and colors, giving them a unique and creative shopping experience.
5. **Comprehensive Product Management for Vendors**: Vendors can efficiently manage their product listings, track inventory in real time, and analyze sales performance, enabling better business decisions and smoother operations.
6. **Order Tracking and Management**: Customers and vendors can track orders from purchase to delivery, providing real-time updates on order status and shipping information.
7. **User Reviews and Social Engagement**: The platform encourages users to share feedback, leave reviews, and engage with other shoppers, creating a sense of community and enhancing trust in product quality.
8. **Secure Checkout and Payment Gateway**: The system ensures a smooth, secure, and user-friendly checkout process with multiple payment options, enhancing customer trust and satisfaction.
9. **Detailed Product Descriptions and Support**: The platform provides comprehensive details on product materials, designs, and maintenance tips, ensuring customers have all the information they need before purchasing.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 FEASIBILITY STUDY

The primary purpose of conducting a feasibility study is to comprehensively assess whether the proposed project can successfully meet the organization's objectives in terms of available resources, labor, and time. This crucial study allows the developers and decision-makers to gain valuable insights into the project's potential viability and prospects. By carefully examining various aspects of the proposed system, such as its impact on the organization, its ability to fulfill user requirements, and the optimal utilization of resources, a feasibility study helps in determining the project's feasibility and potential success.

The assessment of the proposed project's feasibility involves multiple dimensions, each playing acritical role in the decision-making process.

- Technical Feasibility
- Operational Feasibility
- Economic Feasibility

A well-conducted feasibility study provides valuable insights to decision-makers, allowing them to make informed judgments about the project's potential success. It assists in identifying potential risks, challenges, and opportunities associated with the proposed endeavor, enabling stakeholdersto devise effective mitigation strategies.

3.1.1 Economical Feasibility

The economic feasibility of Footland, an online footwear shopping platform, involves assessing the financial viability and investment requirements for its development and maintenance. Key costs include technology infrastructure, such as cloud hosting and secure payment gateways, as well as operational expenses for customer support, server maintenance. Additional investments are required for implementing advanced features like product customization, virtual try-on tools using machine learning and augmented reality, and ensuring data security and regulatory compliance. Revenue generation is projected through product sales commissions, vendor subscription fees, premium services, and advertising or promotional placements.

Footland aligns with the growing consumer demand for personalized and seamless online shopping, making it economically viable. The platform’s core functionalities—such as product management, user authentication, and customer account handling—can be built using web development frameworks and scalable cloud infrastructure.

By leveraging APIs and secure authentication protocols, Footland ensures efficient scaling and user engagement, offering strong potential for long-term revenue growth and sustainability.

3.1.2 Technical Feasibility

The technical feasibility of Footland shows strong potential for successful implementation due to several key factors. The development team has the expertise to utilize modern web technologies, including Django for the backend, HTML/CSS for the front end, and MySQL for database management. Advanced features like virtual try-on tools, product customization, and personalized recommendations are achievable using machine learning and augmented reality frameworks. Scalability is ensured through the use of cloud infrastructure (such as AWS or Google Cloud), which allows the platform to handle increasing traffic and an expanding product catalog efficiently.

Technical assessments confirm that the platform adheres to industry standards for security, performance, and maintainability. The development team can leverage APIs for secure payment processing, user authentication, and inventory management, ensuring the system remains adaptable and capable of scaling as the user base grows. This solid technical foundation ensures that Footland will be a robust, scalable, and secure platform capable of delivering a seamless user experience.

3.1.3 Behavioral Feasibility

Behavioral feasibility for Footland focuses on user and stakeholder acceptance and engagement. User acceptance is critical for the platform's success, as customers must be willing to adopt and engage with features such as virtual try-ons, product customization, and personalized recommendations. The success of the project depends on users finding value in the tailored shopping experience, secure checkout processes, and seamless navigation. Their readiness to adapt to the platform's management tools will determine its smooth operation. Administrative acceptance is also vital, as admins must efficiently manage customer accounts, oversee product listings, and moderate reviews. The administrators' ability to adapt to and embrace the platform's functionalities ensures the system's overall effectiveness. This behavioral feasibility analysis confirms that the platform is well-positioned for widespread acceptance among users, vendors, and administrators.

3.1.4 Feasibility Study Questionnaire

1. Specify the Viewers/Public which is to be involved in the System?

General Visitors, Regular Users

2. List the Modules included in your System?

Admin, Guest Users, Registered Users

3. Identify the users in your project?

Guest Users, Registered Users

4. Who owns the system?

Administrator

5. Which type of categories you sell on this platform?

All footwear's for men,women and kids

6. How often do you expect to change your password for security reasons?

Every 6 months

7. What is your primary reason for purchasing footwear online?

Wider variety

8. System is related to which firm/industry/organization?

E-commerce website

Questionnaire to collect details about the project

1. What types of footwear products are featured on your website (e.g., casual, formal, sports, customizable footwear)?

   Our website offers a wide variety of footwear products, including casual, formal, sports, and customizable options. We feature footwear from multiple brands to cater to diverse customer preferences.

2. What tools or features do you provide for users to interact with products (e.g., reviews, wishlists, product comparisons)?

   Users can interact by leaving reviews and ratings for products, adding items to wishlists for future purchases, and comparing different products to make informed decisions.

3. Can users customize their footwear on your platform?

   Yes, we offer a "Custom Footwear" feature where users can personalize various aspects of their shoes, such as color, design, and materials, to create a unique product tailored to their preferences.

4. How do you handle customer feedback and reviews on products?

   We encourage all feedback, including negative reviews, to improve our product offerings. Our moderation team reviews feedback for authenticity and appropriateness, and we often respond to customer concerns with product recommendations or solutions.

5. How do you ensure your platform is user-friendly and accessible to all users?

We prioritize user experience with a responsive design that works across all devices, intuitive navigation, and accessibility features like alt text for images, easy-to-read fonts, and support for

screen readers.

6. Do you offer personalized product recommendations for users? If so, how do they work?

   Yes, we provide personalized recommendations based on users' browsing history, preferences, and past purchases. Our AI-driven recommendation engine suggests footwear that aligns with their style, brand preferences, and size.

3.1 SYSTEM SPECIFICATION

3.2.1 Hardware Specification

Processor - Ryzen 5

RAM - 8 GB

Hard disk - 500 GB

3.2.2 Software Specification

Front End - HTML, JS, CSS, jQuery, Ajax, Bootstrap

Back End - Python-Django

Database - MYSQL

Client on PC - Windows 7 and above.

Technologies used - JS, HTML5, AJAX,jQuery, CSS

3.3 SOFTWARE DESCRIPTION

3.3.1 DJANGO

Django is a popular and powerful open-source web framework written in Python, designed to facilitate rapid development and maintainable web applications. It follows the Model-View-Template (MVT) architectural pattern, which is similar to the Model-View-Controller (MVC) pattern. Django provides a structured and efficient way to build web applications, offering several key components and features.

At its core, Django includes a robust Object-Relational Mapping (ORM) system that simplifies database interactions, allowing developers to work with Python objects instead of raw SQL queries. It also includes a URL dispatcher for mapping URLs to view functions, an automatic admin interface for managing application data, and a templating engine for creating dynamic andreusable user interfaces.

Django places a strong emphasis on security, with built-in features to protect against common web vulnerabilities. It offers authentication and authorization systems, middleware support for global request and response processing, and compatibility with various databases.

The framework's scalability, extensibility, and a vibrant community of developers make it a popular choice for building web applications, from simple websites to complex, high-traffic platforms. Django's extensive documentation and ecosystem of third-party packages further enhance its appeal for web developers.

3.3.2 MySQL

MySQL is a widely-used open-source relational database management system (RDBMS) developed by Oracle Corporation. It utilizes Structured Query Language (SQL) for managing and manipulating data, providing a reliable and flexible platform for storing and retrieving information. MySQL organizes data into structured tables that can be linked based on common attributes, ensuring efficient data management and integrity. Known for its performance and scalability, MySQL can handle both small-scale and large, complex applications with high volumes of data. It operates across various operating systems, including Windows, Linux, and macOS, and includes robust security features like user authentication and data encryption.

As an open-source platform, MySQL benefits from a large community of developers and users who contribute to its ongoing improvement and provide extensive support. Its combination of speed, reliability, and versatility makes MySQL a popular choice for diverse database needs.

CHAPTER 4
SYSTEM DESIGN

4.1 INTRODUCTION

The initial stage of developing any engineered product or system is the design phase, which involves a creative approach. A well-crafted design plays a critical role in ensuring the successfully functioning of a system. Design is defined as the process of employing various techniques and principles to define a process or system in enough detail to enable its physical realization.This inolves using different methods to describe a machine or system, explaining how it operates, in sufficient detail for its creation. In software development, design is a crucial step that is always present, regardless of the development approach. System design involves creating a blueprint for building a machine or product. Careful software design is essential to ensure optimal performanceand accuracy. During the design phase, the focus shifts from the user to the programmers or those working with the database. The process of creating a system typically involves two key steps: Logical Design and Physical Design

4.2 UML DIAGRAM

UML, which stands for Unified Modeling Language, is a standardized language used for specifying, visualizing, constructing, and documenting the elements of software systems. The Object Management Group (OMG) is responsible for the creation of UML, with the initial draft of the UML 1.0 specification presented to OMG in January 1997. Unlike common programming languages such as C++, Java, or COBOL, UML is not a programming language itself. Instead, it is a graphical language that serves as a tool for creating software blueprints.

UML is a versatile and general-purpose visual modeling language that facilitates the visualization, specification, construction, and documentation of software systems. While its primary use is in modeling software systems, UML's applications are not limited to this domain. It can also be employed to represent and understand processes in various contexts, including non-software scenarios like manufacturing unit processes.

It's important to note that UML is not a programming language, but it can be utilized with tools that generate code in different programming languages based on UML diagrams. UML encompasses nine core diagrams that aid in representing various aspects of a system.

- Class diagram
- Object diagram
- Use case diagram
- Sequence diagram

- Activity diagram
- State chart diagram
- Deployment diagram
- Component diagram

4.2.1.1.1 USE CASE DIAGRAM

A use case is a tool for understanding a system's requirements and organizing them, especially in the context of creating or using something like a product delivery website. These tools are represented using "use case" diagrams within the Unified Modeling Language, a standardized wayof creating models for real-world things and systems.
A use case diagram consists of these main elements:

- The boundary, which delineates the system and distinguishes it from its surroundings.
- Actors, representing individuals or entities playing specific roles within the system.
- The interactions between different people or elements in specific scenarios or problems.
- The primary purpose of use case diagrams is to document a system's functional specifications. To create an effective use case diagram, certain guidelines must be followed:
- Providing clear and meaningful names for use cases and actors.
- Ensuring that the relationships and dependencies are well-defined.
- Including only necessary relationships for the diagram's clarity.
- Utilizing explanatory notes when needed to clarify essential details.

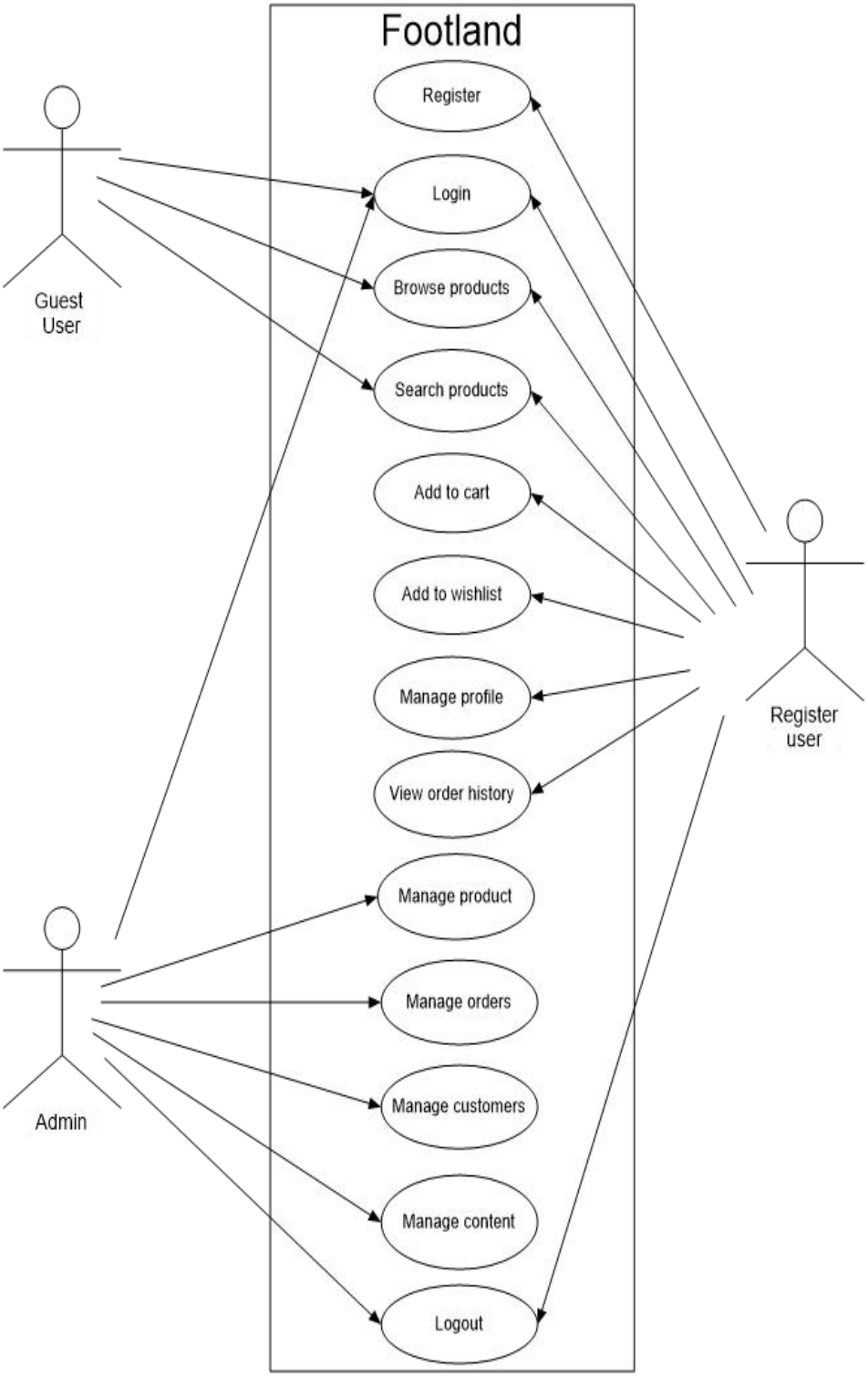


Fig4.2.1.1.1: Use Case Diagram

4.2.1 SEQUENCE DIAGRAM

A sequence diagram illustrates the specific order in which objects interact with each other, showcasing the sequential flow of events. This type of diagram is also known as event diagrams or event scenarios. Sequence diagrams serve the purpose of elucidating how various components of a system collaborate and the precise sequence in which these actions occur. These diagrams find frequent application among business professionals and software developers, aiding in the understanding and depiction of requirements for both new and existing systems.

Sequence Diagram Notations:

i. Actors - Within a UML diagram, actors represent individuals who utilize the system and its components. Actors are not depicted within the UML diagram as they exist outside the system being modeled. They serve as role-players in a story, encompassing people and external entities. In a UML diagram, actors are represented by simple stick figures. It's possible to depict multiple individuals in a diagram that portrays the sequential progression of events.

ii. Lifelines - In a sequence diagram, each element is presented as a lifeline, with lifeline components positioned at the top of the diagram.

iii. Messages - Communication between objects is achieved through the exchange of messages, with messages being arranged sequentially on the lifeline. Arrows are employedto represent messages, forming the core structure of a sequence diagram.

iv. Guards - Within the UML, guards are employed to denote various conditions. They are used to restrict messages in the event that specific conditions are met, providing softwaredevelopers with insights into the rules governing a system or process.

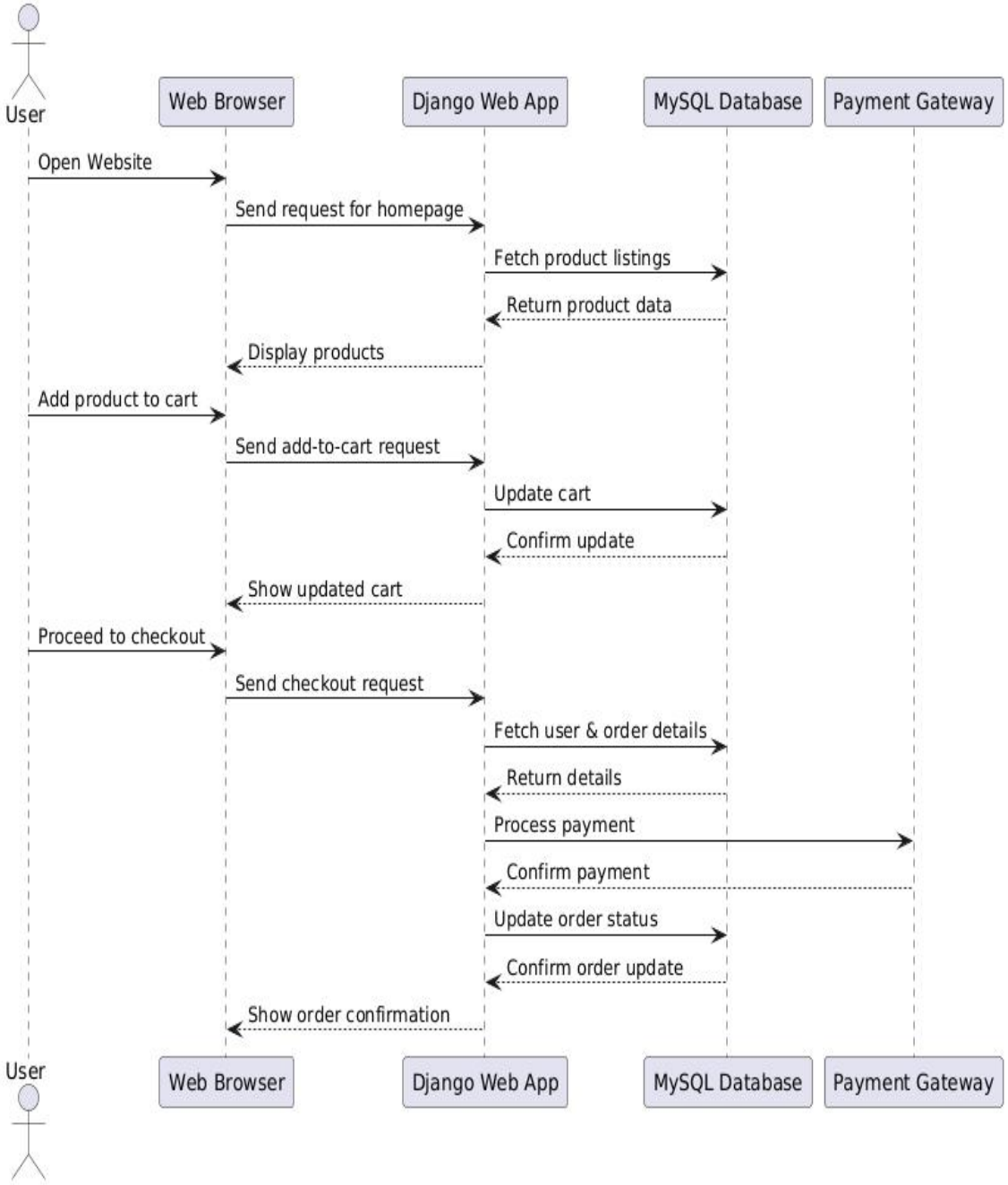


Fig4.2.1: Sequence Diagram

4.2.2 State Chart Diagram

A state machine diagram, also known as a state chart, visually represents the various states an object undergoes within a system and the sequence in which these states are traversed. It serves as a record of the system's behavior, illustrating the collaborative functioning of a group of entities, whether it's a team, a collection of students, a large assembly, or an entire organization. State machine diagrams are a valuable method for depicting the interactions of diverse components within a system, outlining how objects evolve in response to events and elucidating the diverse conditions that each entity or component can inhabit.

Notations within a state machine diagram encompass:

- Initial state: Symbolized by a black circle, this signifies the commencement of a process.
- Final state: Representing the conclusion of a process, this is denoted by a filled circle within another circle.
- Decision box: Shaped like a diamond, it aids in decision-making by considering the evaluation of a guard.
- Transition: Whenever a change in authority or state occurs due to an event, it is termed atransition. Transitions are depicted as arrows with labels indicating the triggering event.
- State box: It portrays the condition or state of an element within a group at a specificmoment. These are typically represented by rectangles with rounded corners.

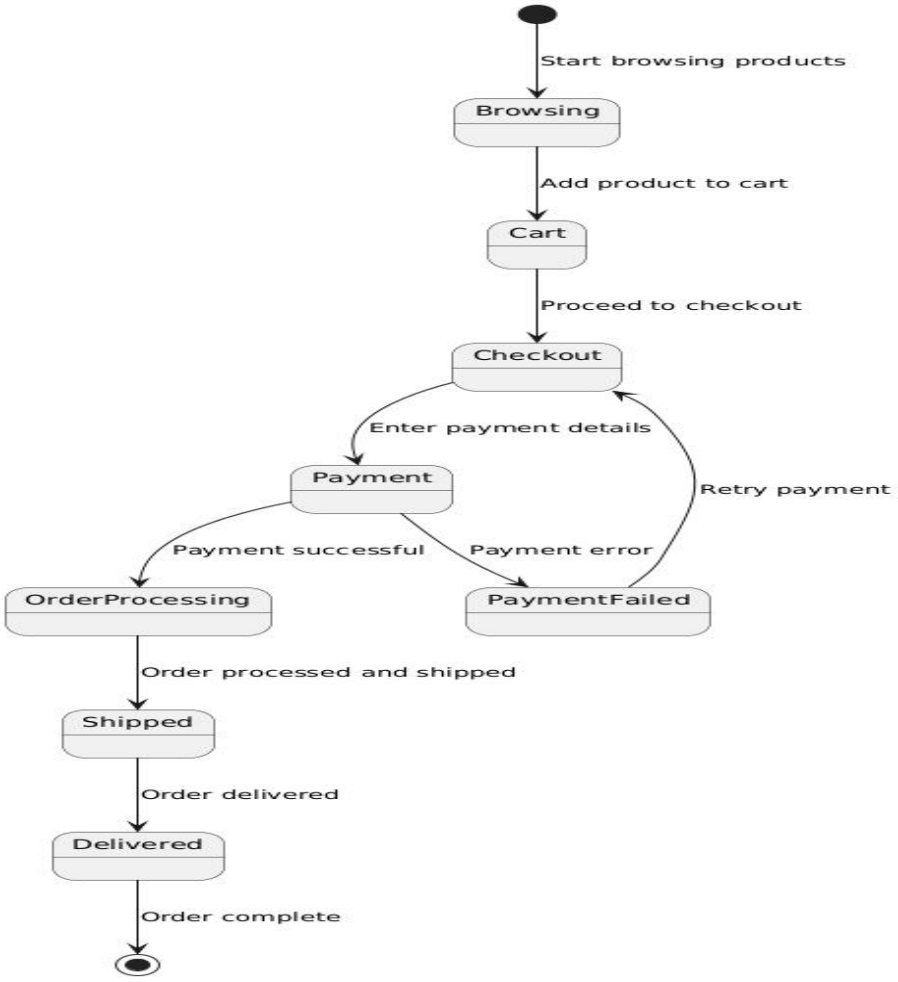


Fig 4.2.2: State Chart Diagram

4.2.3Activity Diagram

An activity diagram is a visual representation of how events unfold simultaneously or sequentially. It aids in understanding the flow of activities, emphasizing the progression from one task to the next. Activity diagrams focus on the order in which tasks occur and can depict various types of flows, including sequential, parallel, and alternative paths. To facilitate these flows, activity diagrams incorporate elements such as forks and join nodes, aligning with the concept of illustrating the functioning of a system in a specific manner.

Key Components of an Activity Diagram include:

a) Activities: Activities group behaviors into one or more actions, forming a network of interconnected steps. Lines between these actions outline the step-by-step sequence of events. Actions encompass tasks, controlling elements, and resources utilized in the process.

b) Activity Partition/Swim Lane: Swim lanes are used to categorize similar tasks into rows orcolumns, enhancing modularity in the activity diagram. They can be arranged vertically orhorizontally, though they are not mandatory for every activity diagram.

c) Forks: Fork nodes enable the simultaneous execution of different segments of a task. They represent a point where one input transforms into multiple outputs, resembling the diversefactors influencing a decision.

d) Join Nodes: Join nodes are distinct from fork nodes and employ a Logical AND operationto ensure that all incoming data flows converge to a single point, promoting synchronization.

Notations in an Activity Diagram include:

- Initial State: Depicting the beginning or first step in the process.
- Final State: Signifying the completion of all actions with no further progress.
- Decision Box: Ensuring that activities follow a specific path.
- Action Box: Representing the specific tasks or actions to be performed in the process.

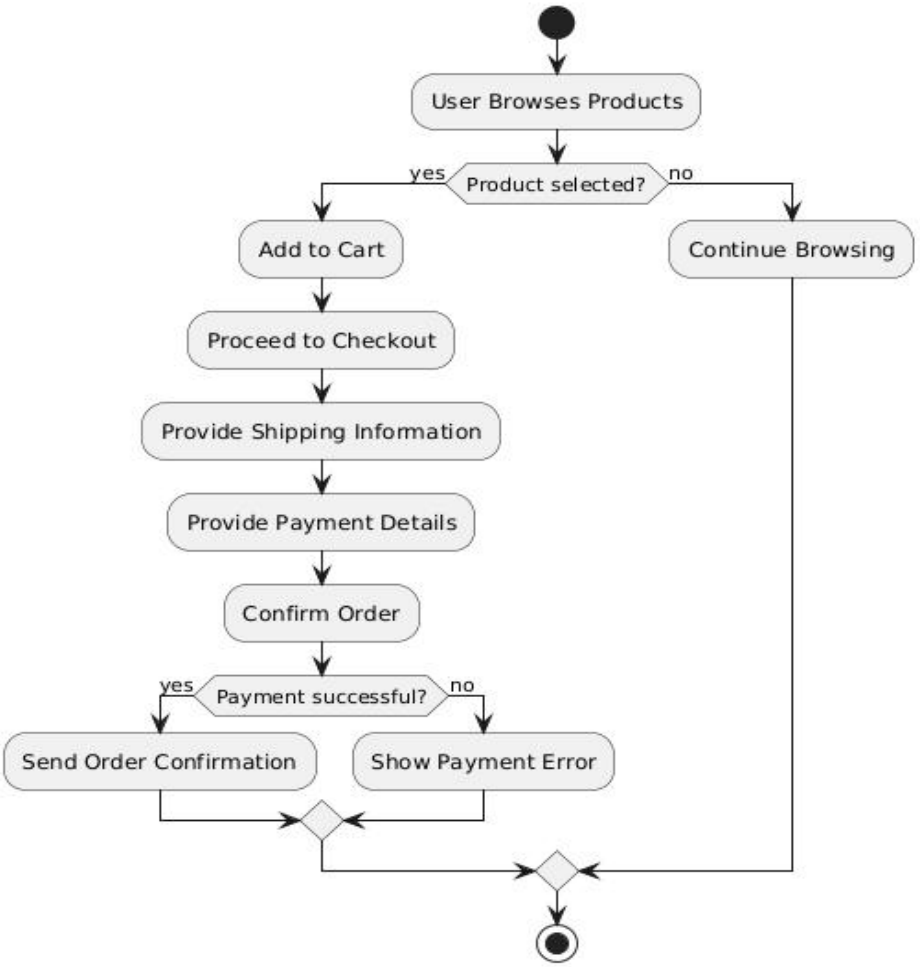


Fig 4.2.3: Activity Diagram

4.2.4 Class Diagram

A class diagram serves as a static blueprint for an application, illustrating its components and their relationships when the system is in a dormant state. It provides insights into the system's structure, showcasing the various elements it comprises and how they interact. In essence, a class diagram acts as a visual guide for software development, aiding in the creation of functional applications.

Key Aspects of a Class Diagram:

- System Overview: A class diagram offers a high-level representation of a software system, presenting its constituent parts, associations, and collaborative dynamics. It serves as an organizational framework for elements such as names, attributes, and methods, simplifyingthe software development process.
- Structural Visualization: The class diagram is a structural diagram that combines classes, associations, and constraints to define the system's architecture.

Components of a Class Diagram:

The class diagram comprises three primary sections:

- Upper Section: This top segment features the class name, representing a group of objects that share common attributes, behaviors, and roles. Guidelines for displaying groups of objects include capitalizing the initial letter of the class name, positioning it in the center, using bold lettering, and employing slanted writing style for abstract class titles.
- Middle Section: In this part, the class's attributes are detailed, including their visibility indicators, denoted as public (+), private (-), protected (#), or package (~).
- Lower Section: The lower section elaborates on the class's methods or operations, presented in a list format with each method on a separate line. It outlines how the class interacts with data.

In UML, relationships within a class diagram fall into three categories:

- Dependency: Signifying the influence of one element's changes on another.
- Generalization: Representing a hierarchical relationship where one class acts as a parent,and another serves as its child.
- Association: Indicating connections between elements.
- Multiplicity: Defining constraints on the number of instances allowed to possess specificcharacteristics, with one being the default value when not specified.
- Aggregation: An aggregation is a group that is a part of a relationship called association.

Composition: "Composition" is like a smaller part of "aggregation.". This describes how aparent and child need each other, so if one is taken away, the other won't work anymore.

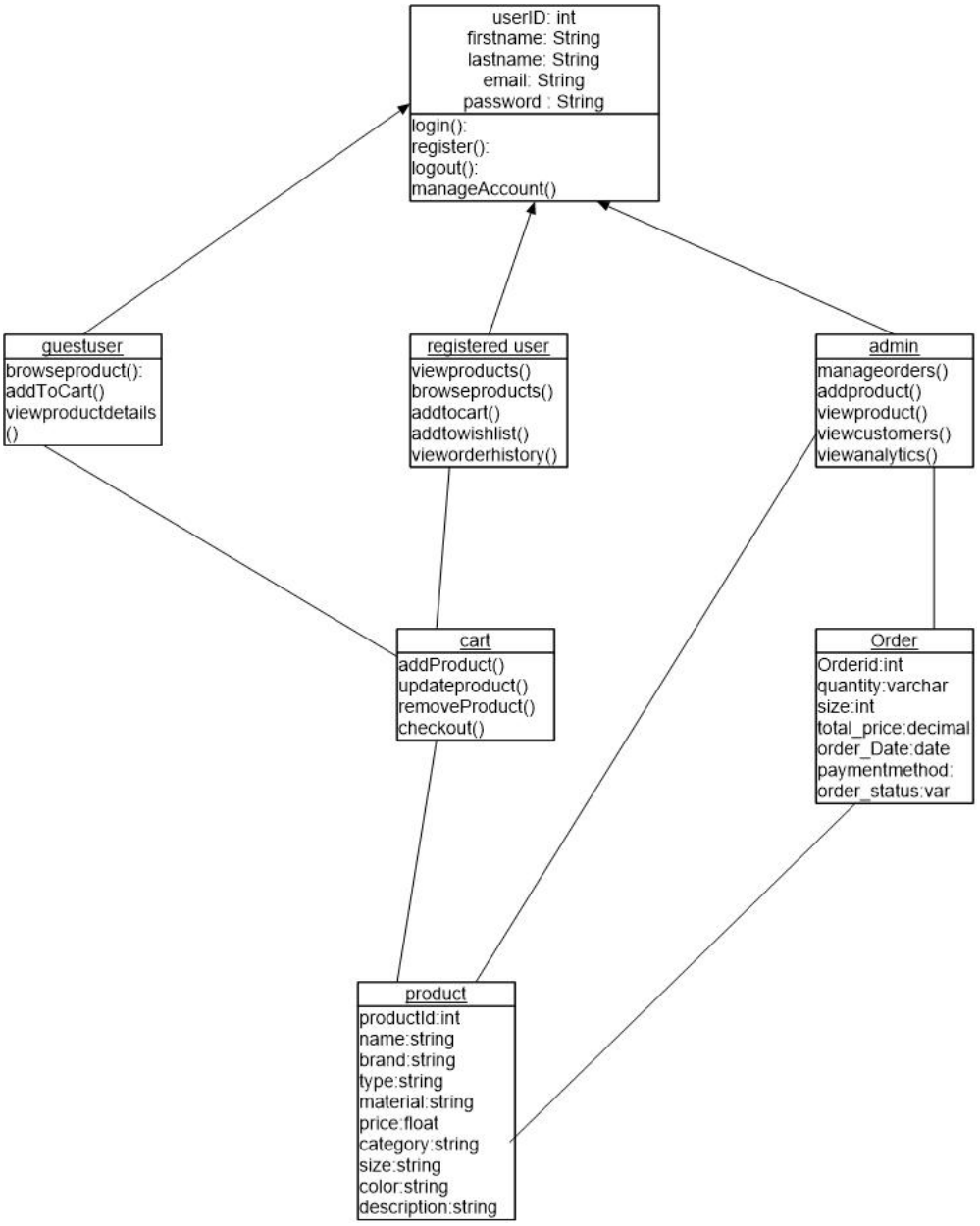


Fig 4.2.4: Class Diagram

4.2.5 Object Diagram

Object diagrams are derived from class diagrams and rely on them to provide a visual representation. They offer an illustration of a collection of objects related to a particular class. Object diagrams provide a snapshot of objects in an object-oriented system at a specific point in time.

Object diagrams and class diagrams share similarities, but they also have distinctions. Class diagrams are more generalized and do not portray specific objects. This abstraction in class diagrams simplifies the comprehension of a system's functionality and structure.

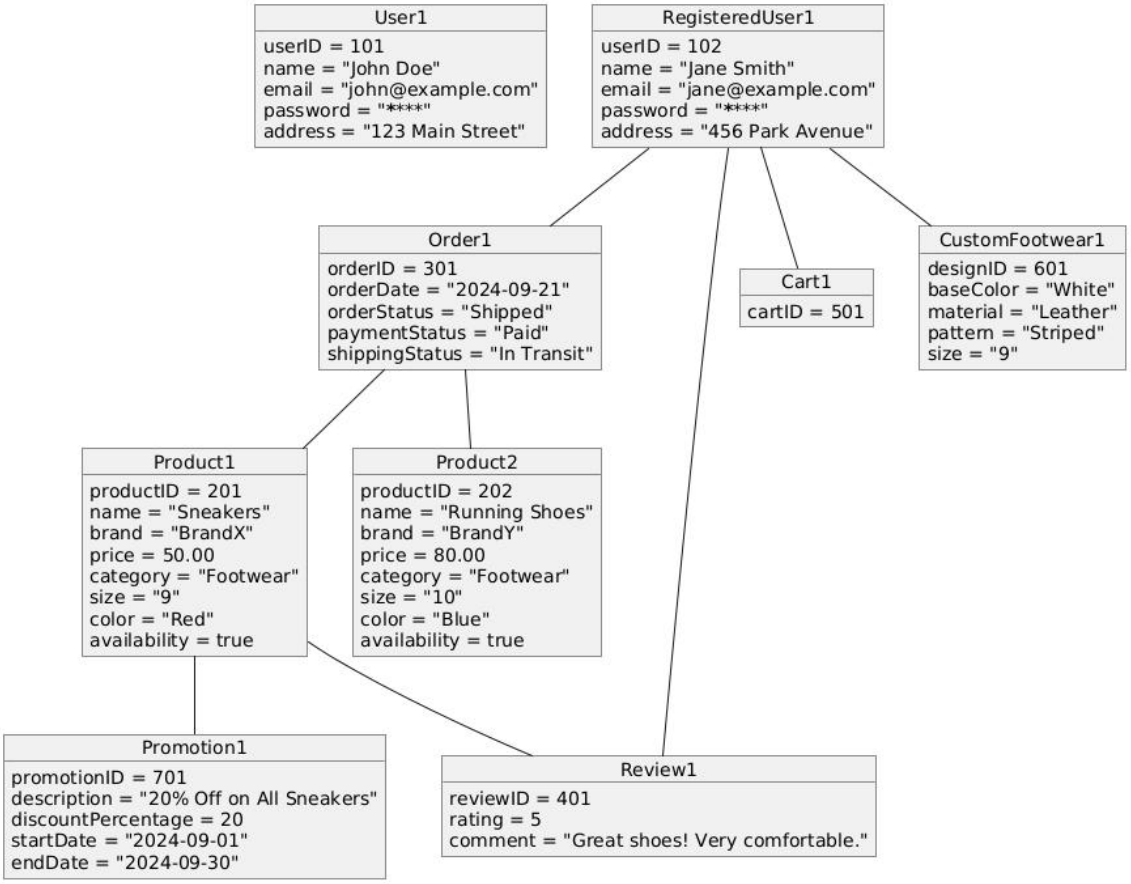


Fig 4.2.5: Object Diagram

4.2.6 Component Diagram

A component diagram serves the purpose of breaking down a complex system that utilizes objects into more manageable segments. It offers a visual representation of the system, showcasing its internal components such as programs, documents, and tools within the nodes.

This diagram elucidates the connections and organization of elements within a system, resulting in the creation of a usable system.

In the context of a component diagram, a component refers to a system part that can be modified and operates independently. It retains the secrecy of its internal operations and requires a specific method to execute a task, resembling a concealed box that functions only when operated correctly.

Notation for a Component Diagram includes:

- A component
- A node

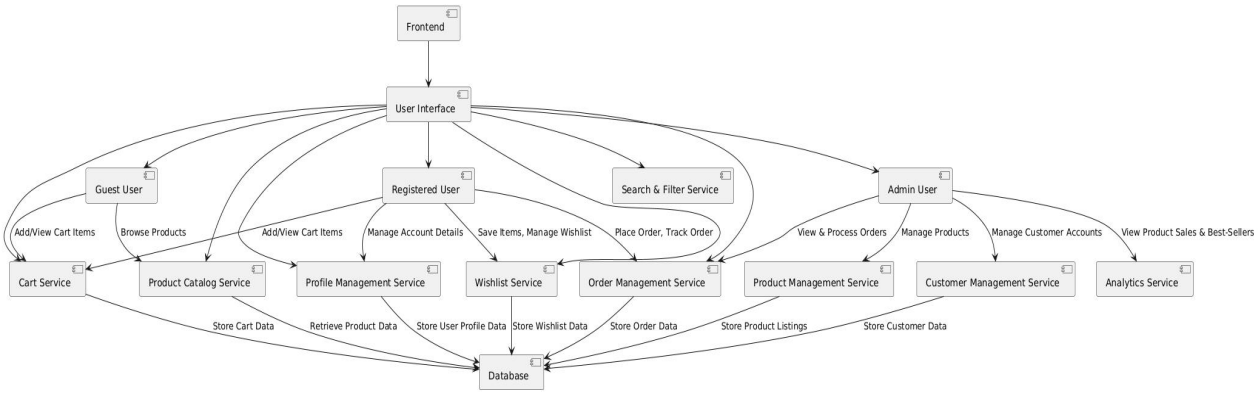


Fig 4.2.6: Compontent Diagram

4.2.8 Deployment Diagram

A deployment diagram provides a visual representation of how software is positioned on physical computers or servers. It depicts the static view of the system, emphasizing the arrangement of nodes and their connections.

This type of diagram delves into the process of placing programs on computers, elucidating how software is constructed to align with the physical computer system.

Notations in a Deployment Diagram include:

- A component
- An artifact
- An interface
- A node

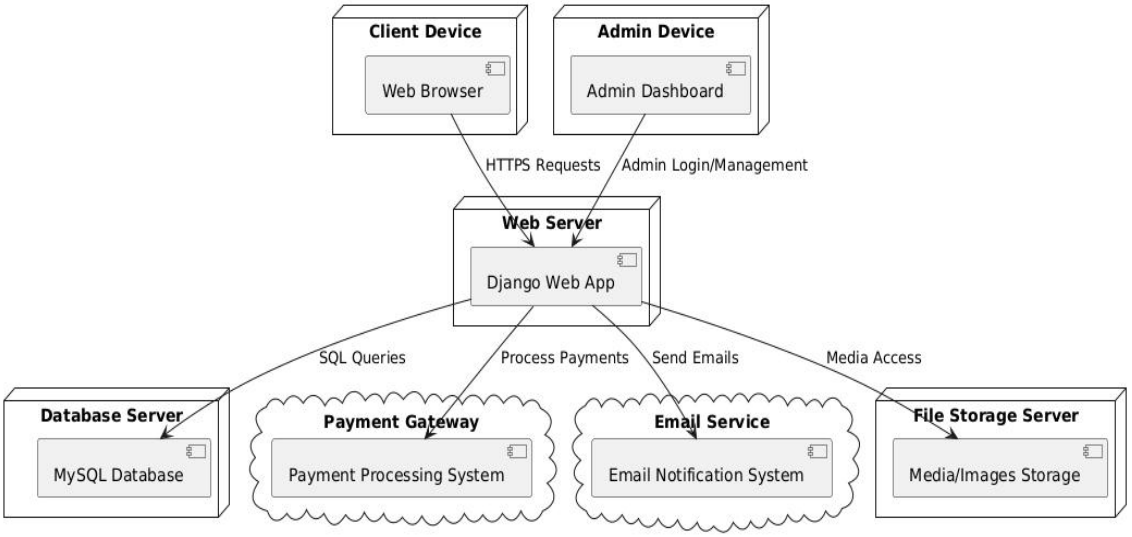


Fig 4.2.8: Deployment Diagram

4.3 USER INTERFACE DESIGN USING FIGMA

4.3.1 Form Name: Login Page

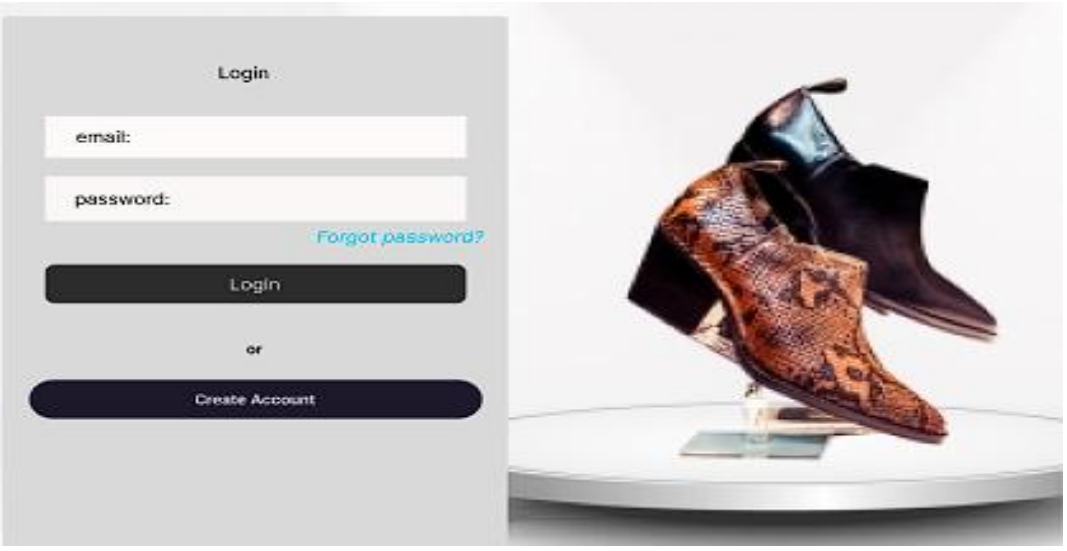


Figure:4.3.1 : Login Page

4.3.2Form Name: Registration Page

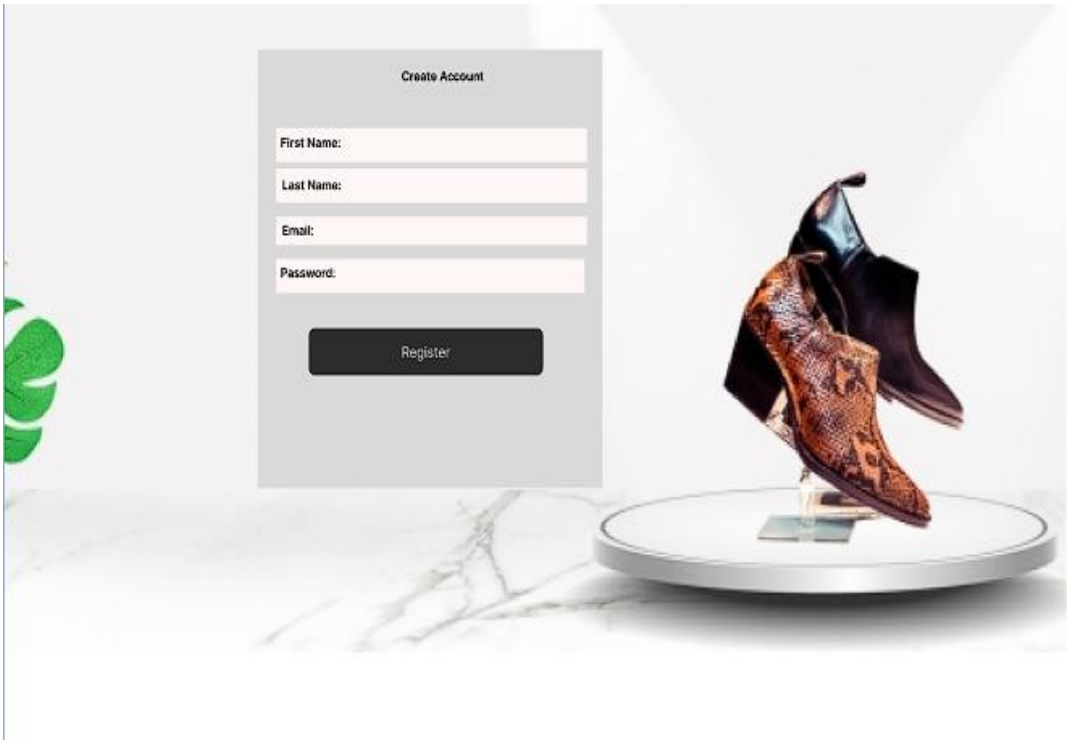


Figure 4.3.2: Registration Page

4.3.3 Form Name: Home Page

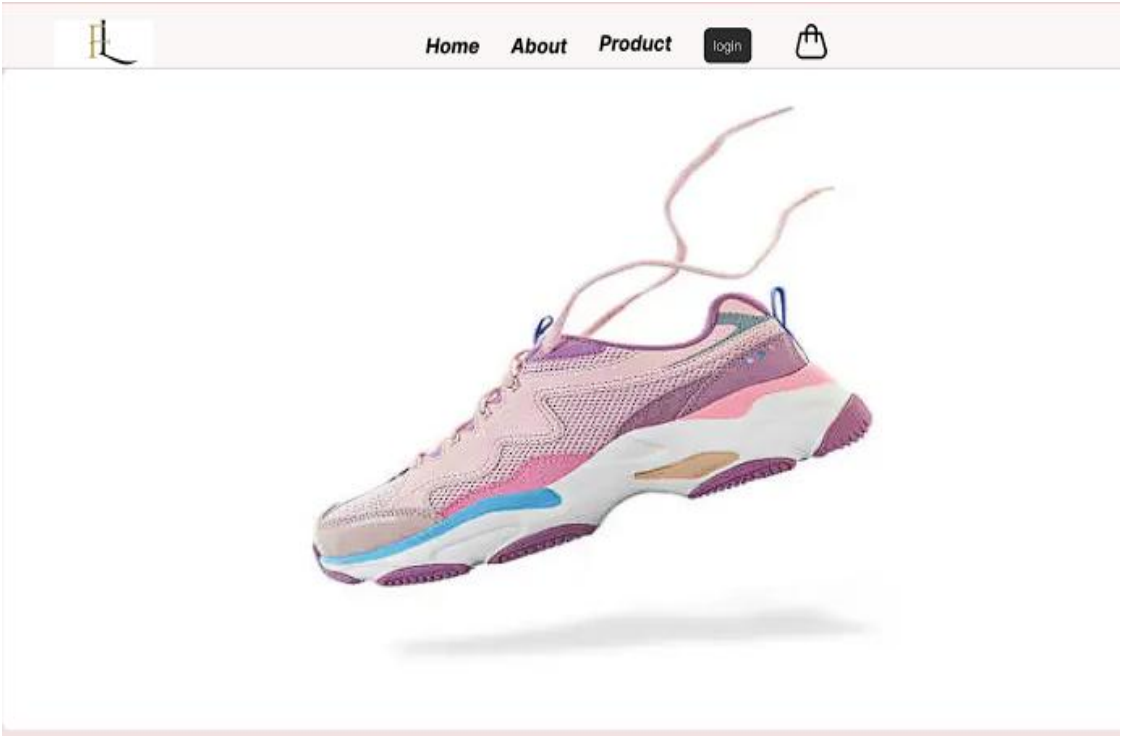


Figure:4.3.3: Home Page

4.3.4 Form Name: Product Page

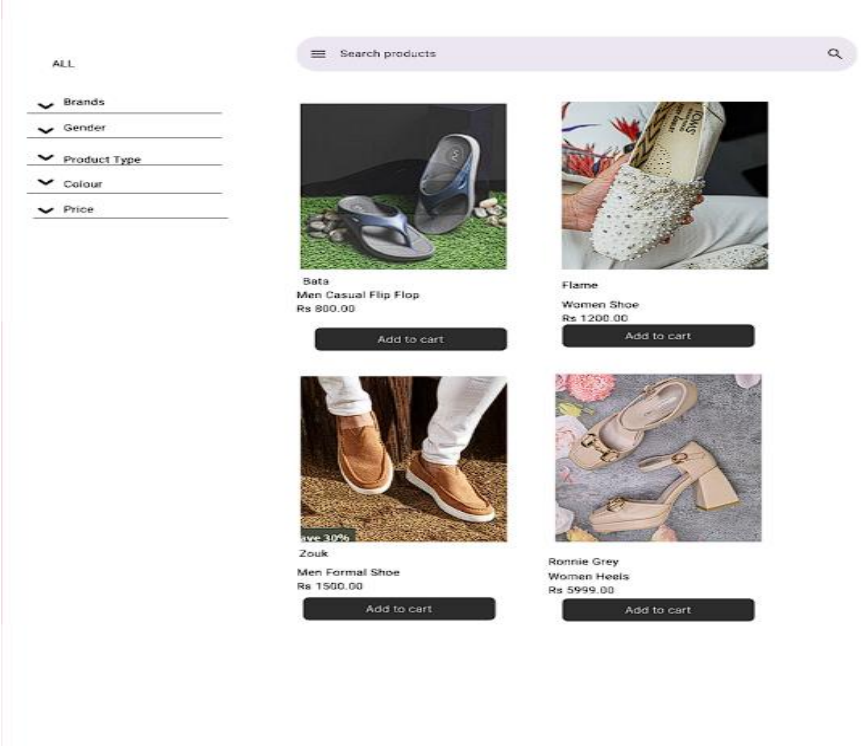


Figure:4.3.4: Product Page

4.3.5 Form Name: Product Details Page

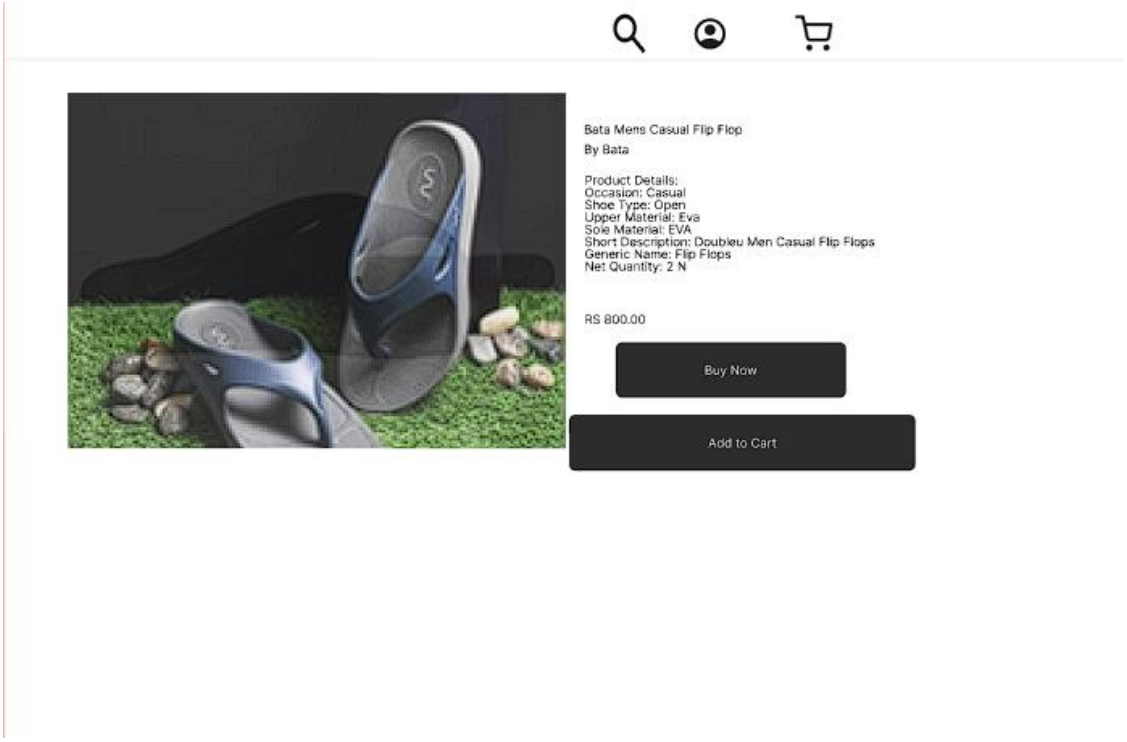


Figure 4.3.5: Product Details Page

4.3.6 Form Name: Cart Page

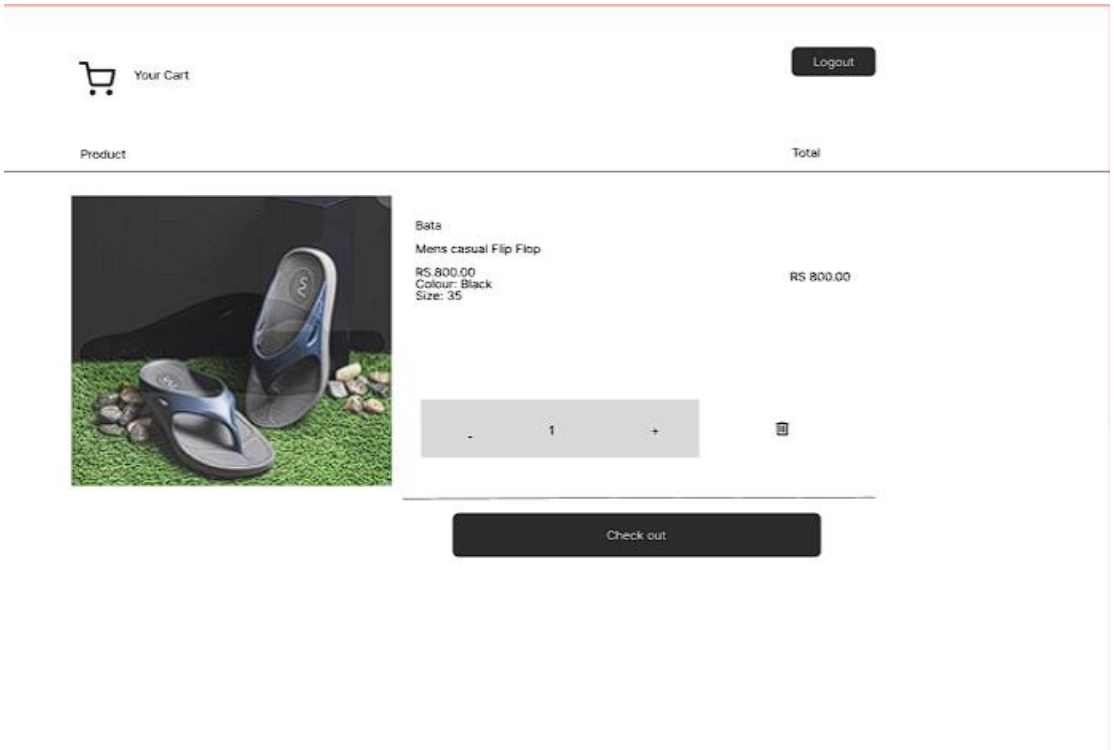


Figure 4.3.6: Cart Page

4.4 DATABASE DESIGN

4.4.1 Relational Database Management System (RDBMS)

A database is a system designed to store and facilitate the retrieval and utilization of information. The integrity and security of the data within a database are of paramount importance.

Creating a database involves two essential steps. Initially, it is crucial to understand the user's requirements and establish a database structure that aligns with their needs. The first step entails devising an organizational plan for the information, which is not reliant on any particular computer program.Subsequently, this plan is employed to craft a design tailored to the specific computer program thatwill be employed to construct the database system. This phase is centered on determining how datawill be stored within the chosen computer program.

Key aspects of database development include:

- Data Integrity: Ensuring the accuracy and consistency of data stored within the database.
- Data Independence: The ability to modify the data storage structure without affecting theapplication's access to the data.

4.4.2 Normalization

A way of showing a database as a group of connections is called a relational model. Every relationship is like a chart of information or a group of data. In the formal way of talking about tables, a row is called a tuple, a column header is called an attribute, and the table is called a relation. A bunch of tables with special names make up a relational database. A row in a story shows a groupof things that are connected.

Relations, Domains & Attributes

Tables serve as containers for organized and related information. Within a table, the rows are referred to as tuples, and a tuple is essentially an ordered list containing n items. Columns in a tableare synonymous with characteristics, each representing a specific aspect of the data.

In a relational database, tables are interconnected, ensuring coherence and meaningful associationsbetween different sets of data. This relational structure forms the foundation of a well-structured database.

A domain can be thought of as a collection of fundamental values with a shared source or data type. Naming domains is a helpful practice as it aids in comprehending the significance of the values theyencompass. It's essential to recognize that values within a domain are indivisible

Table relationships are established using keys, with the primary key and foreign key being the primary types. Entity Integrity and Referential Integrity are fundamental principles that guide these relationships.Entity Integrity mandates that no Primary Key should contain null values, reinforcing the unique and integral nature of primary keys in database relationships.

Normalization is the process of organizing data to ensure its efficient storage and to minimize redundancy while maintaining accuracy and reliability. This approach involves eliminating unnecessary columns and breaking down large tables into smaller, more manageable parts. Normalization helps prevent errors when adding, removing, or modifying data. In data modeling, two key concepts are integral to its normal form: keys and relationships.

Keys serve as unique identifiers to distinguish one row from all others in a table. There are two maintypes of keys: the primary key, which groups similar records within a table, and the foreign key, which acts as a special code to identify specific records in another table.

1. **First Normal Form (1NF):** This rule dictates that an attribute can only contain a single value that cannot be further divided. Each value in a tuple must match the attribute's type. In 1NF, tables cannot contain other tables or have tables as part of their data. This form ensures that values are indivisible and represent a single entity. Achieving 1NF often involves organizing data into separate tables, with each table having a designated key based on project requirements. New relationships are established for various data categories or related groups, eliminating redundant information. A relationship adhering to primary key constraints alone is termed the first normal form.
2. **Second Normal Form (2NF)**: In simpler terms, 2NF stipulates that no additional information should be associated with only part of the primary information used for data organization. This involves breaking down the information and creating separate groupings for each part along with its related details. It's essential to maintain a connection between the original primary key and any data dependent on it. This step helps remove data that relies on only a portion of the key. When a set of information has a primary means of identifying each item, and all other details depend solely on that primary means, it is considered to be in the second normal form.

3. **Third Normal Form (3NF)** is a critical concept in database normalization, aiming to achieve table independence and control over attributes. In 3NF, a table should not contain columns thatdepend on other non-key columns, ensuring that each column is functionally dependent only on the primary key. It breaks down relationships involving non-key attributes to eliminate dependencies on attributes not part of the primary key. To meet the criteria for 3NF, data must already be in the Second Normal Form (2NF), and non-key attributes should not rely on other non-key attributes within the same table, avoiding transitive dependencies. This process enhances data integrity, reduces redundancy, and optimizes database structure and organization.

4.4.3 Sanitization

Django incorporates various in-built mechanisms for data sanitization. To begin with, it provides adiverse range of field types that come with automatic validation and sanitization capabilities. For instance, the CharField performs automatic validation and cleaning of text input, while the EmailField ensures that email addresses adhere to the correct format. These field types serve as protective measures to prevent the storage of malicious or invalid data in the database.

Moreover, Django strongly advocates for the utilization of form validation to sanitize user input. Django's forms are equipped with predefined validation methods and validators that can be appliedto form fields. These validators execute a range of checks and cleansing operations, such as confirming that numerical values fall within specified ranges or verifying that uploaded files adhereto specific formats. These combined features in Django promote data integrity and security, makingit a reliable choice for web application development.

4.4.4 Indexing

The index keeps track of a certain piece of information or group of information, arranged in order of its value. Arranging the index entries helps find things quickly and easily that match exactly or are within a certain range. Indexes make it easy to find information in a database without having to search through every record every time the database is used. An index is like a roadmap for findinginformation in a database. It helps you look up data quickly and also makes it easy to find records that are in a certain order. It can be based on one or more columns in the table

4.5 TABLE DESIGN

1. Tbl_user_user_registration

Primary key:**user_user_registration**_id

No:	Field name	Datatype (Size)	Key Constraints	Description of the field
1	id	int	Primary Key	Registered user id
2	First_name	varchar	not null	Registered first name
3	Last_name	varchar	Not null	Registered last name
4	email	varchar	Not null	Registered eamil
5	password	varchar	Not null	Registered password
6	User_type	varchar	Not null	Which user is logined
7	Is_active	boolean	Not null	Check the active status

2. Tbl_user_product

Primary key: userproduct_id

No:	Field name	Datatype (Size)	Key Constraints	Description of the field
1	id	int	Primary Key	product id
2	name	varchar	not null	product name
3	description	varchar	Not null	Product description
4	material	varchar	Not null	Product material
5	brand	varchar	Not null	Product brand
6	type	varchar	Not null	Which type is the product
7	price	decimal	Not null	Price of the product
8	Stock_quantity	int	Not null	Stock of the product
9	image	image	Not null	Product image
10	size	varchar	Not null	Size of the product
11	color	varchar	Not null	Color of each product
12	category	varchar	Not null	Which category ie for men,women,kids

3. Tbl_user_userprofile

Primary key: userprofile_id

Foreign Key: **User_id** references **tbl_User_registration**

No:	Field name	Datatype (Size)	Key Constraints	Description of the field
1	id	int	Primary Key	User profile id
2	address	varchar	not null	Registered user address
3	pincode	varchar	Not null	Registered user pincode
4	phone	varchar	Not null	Registered user phone
5	state	varchar	Not null	Registered user state
6	street	varchar	Not null	Which user is street
7	House_no	varchar	Not null	Registered user house_no
8	User_id	int	Foregin key	Id from the user_registration

4. Tbl_user_cart

Primary key: cart_id

Foreign Key: **User_id** references **tbl_User_registration**, product_id references **tbl_products**

No:	Field name	Datatype (Size)	Key Constraints	Description of the field
1	id	int	Primary Key	Id for cart
2	User_id	int	Foregin key	Id that from the user_registration
3	Product_id	int	Foregin key	Id from the product
4	quantity	varchar	Not null	Quantity of the product
5	size	varchar	Not null	Size of the product
6	Saved_for_later	boolean	Not null	Saved_for_later

5. Tbl_user_saveditem

Primary key: saved_item_id

Foreign Key: **User_id** references **tbl_User_registration**, product_id references **tbl_products**

No:	Field name	Datatype (Size)	Key Constraints	Description of the field
1	id	int	Primary Key	Id for saved_for _later
2	Date_saved	date	Not null	Date for saved item
3	Product_id	int	Foregin key	Id from the product
4	User_id	int	Foregin key	Id that from the user_registration

6. Tbl_user_wishlist

Primary key: wishlist_id

Foreign Key: **User_id** references **tbl_User_registration**, product_id references **tbl_products**

No:	Field name	Datatype (Size)	Key Constraints	Description of the field
1	id	int	Primary Key	Id for saved_for _later
4	User_id	int	Foregin key	Id that from the user_registration
3	Product_id	int	Foregin key	Id from the product

7 Tbl_user_order

Primary key: order_id

Foreign Key: **User_id** references **tbl_User_registration**, product_id references **tbl_products**

No:	Field name	Datatype (Size)	Key Constraints	Description of the field
1	id	int	Primary Key	Id for cart
2	User_id	int	Foregin key	Id that from the user_registration
3	Product_id	int	Foregin key	Id from the product
4	quantity	varchar	Not null	Quantity of the product
5	size	int	Not null	Size of the product
6	Total_price	decimal	Not null	Price of the product
7	Order_date	date	Not null	Date in which the item is orderd
8	Payment_method	varchar	Not null	Payment_method
9	Order_status	varchar	Not null	Status of the order

CHAPTER 5
SYSTEM TESTING

5.1 INTRODUCTION

Software testing is an essential procedure employed to verify if a computer program functions as intended. It is conducted to ensure that the software performs its designated tasks accurately and complies with the prescribed requirements and standards. Validation is the process of inspecting and assessing software to confirm its adherence to the specified criteria. Software testing is a method for assessing the performance of a program, often in conjunction with techniques like code inspection and program walk throughs. Validation ensures that the software aligns with the user's expectations and requirements.

Several principles and objectives guide the process of software testing, including:

1. Testing is the practice of executing a program with the primary aim of identifying errors.
2. An effective test case is one that has a high likelihood of uncovering previously undiscovered errors.
3. A successful test is one that exposes previously undiscovered errors.

When a test case operates effectively and accomplishes its objectives, it can detect flaws within the software. This demonstrates that the computer program is functioning as intended and is performing well. The process of evaluating a computer program encompasses three primary aspects:

1. Correctness assessment
2. Evaluation of implementation efficiency
3. Examination of computational complexity

5.2TEST PLAN

A test plan serves as a comprehensive set of instructions for conducting various types of tests. It can be likened to a map that outlines the steps to follow when evaluating a computer program. Software developers create instructions for both using and organizing the necessary information for the program's proper functionality. They ensure that each component of the program performs its intended functions. To ensure the thorough testing of the software, a group known as the ITG (Information Technology Group) is responsible for verifying its functionality, instead of relying solely on the software's creators for testing.

The objectives of testing should be clearly defined and measurable. A well-structured test plan should encompass details about the frequency of failures, the associated repair costs, the occurrence rate of issues, and the time required for complete testing. The levels of testing typically include:

- Unit testing
- Integration testing
- Data validation testing
- Output testing

5.2.1 Unit Testing

Unit testing checks the smallest part of a software design - the software component or module. Testing important control paths within a module using the design guide to find errors. This means how difficult the tests are for each small part of a program and what parts of the program haven't been tested yet. Unit testing is a type of testing that looks at how the code works inside and can be done at the same time for different parts of the program.

Before starting any other test, we need to check if the data flows correctly between different parts of the computer program. If the information doesn't move in and out correctly, all other checks are pointless. When designing something, it's important to think about what could go wrong and makea plan for how to deal with those problems. This can mean redirecting the process or stopping it completely.

The Sell-Soft System was tested by looking at each part by itself and trying different tests on it. Some mistakes in the design of the modules were discovered and then fixed. After writing the instructions for different parts, each part is checked and tried out separately. We got rid of extra code and made sure everything works the way it should.

5.2.2 Integration Testing

Integration testing is a critical process in software development that involves constructing a program while simultaneously identifying errors in the interaction between different program components. The primary objective is to utilize tested individual parts and assemble them into a program according to the initial plan. This comprehensive testing approach assesses the entire program to ensure its proper and correct functionality.

As issues are identified and resolved during integration testing, it's not uncommon for new problems to surface, leading to an ongoing cycle of testing and refinement. After each individual component of the system is thoroughly examined, these components are integrated to ensure they function harmoniously. Additionally, efforts are made to standardize all programs to ensure uniformity rather than having disparate version.

5.2.3 Validation Testing or System Testing

Integration testing is a critical process in software development that involves constructing a programwhile simultaneously identifying errors in the interaction between different program components. The primary objective is to utilize tested individual parts and assemble them into a program according to the initial plan. This comprehensive testing approach assesses the entire program to ensure its proper and correct functionality.

As issues are identified and resolved during integration testing, it's not uncommon for new problems to surface, leading to an ongoing cycle of testing and refinement. After each individual component of the system is thoroughly examined, these components are integrated to ensure they function harmoniously. Additionally, efforts are made to standardize all programs to ensure uniformity ratherthan having disparate versions

5.2.4 Output Testing or User Acceptance Testing

System testing is conducted to assess user satisfaction and alignment with the company's requirements. During the development or update of a computer program, it's essential to maintain aconnection with the end-users. This connection is established through the following elements:

- Input Screen Designs.
- Output Screen Designs.

To perform system testing, various types of data are utilized. The preparation of test data plays a crucial role in this phase. Once the test data is gathered, it is used to evaluate the system under investigation. When issues are identified during this testing, they are addressed by following established procedures. Records of these corrections are maintained for future reference andimprovement. This process ensures that the system functions effectively and meets the needs of both users and the organization.

5.2.5 Automation Testing

Automated testing is a method employed to verify that software functions correctly and complies with established standards before it is put into official use. This type of testing relies on written instructions that are executed by testing tools. Specifically, UI automation testing involves the use of specialized tools to automate the testing process. Instead of relying on manual interactions where individuals click through the application to ensure its proper functioning, scripts are created to automate these tests for various scenarios. Automating testing is particularly valuable when it is necessary to conduct the same test across multiple computers simultaneously, streamlining the testing process and ensuring consistency.

5.2.6 Selenium Testing

Selenium is a valuable and free tool designed for automating website testing. It plays a crucial role for web developers as it simplifies the testing process. Selenium automation testing refers to the practice of using Selenium for this purpose. Selenium isn't just a single tool; it's a collection of tools, each serving distinct functions in the realm of automation testing. Manual testing is a necessary aspect of application development, but it can be monotonous and repetitive. To alleviate these challenges, Jason Huggins, an employee at Thought Works, devised a method for automating testingprocedures, replacing manual tasks. He initially created a tool named the JavaScript TestRunner to facilitate automated website testing, and in 2004, it was rebranded as Selenium.

Test Case 1 – Reader Login

Code

```
package definitions;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import io.cucumber.java.en.And;
import io.cucumber.java.en.Given;
import io.cucumber.java.en.Then;
import io.cucumber.java.en.When;
public class step_defenition {
   WebDriver driver=null;
   @Given("browser is open")
   public void browser_is_open() {
      System.out.println("Inside step-Browser is open");
      System.setProperty("webdriver.gecko.marionette","C:\\Users\\bibin\\eclipse-
workspace\\sample\\src\\test\\resources\\drivers\\geckodriver.exe");
      driver=new FirefoxDriver();
      driver.manage().window().maximize();
   }
   @And("user is on login page")
   public void user_is_on_login_page() throws Exception {
      driver.navigate().to("http://127.0.0.1:8000/login/");
      Thread.sleep(2000);
   }
   @When("user enters username and password")
public void user_enters_username_and_password() throws Throwable{
      driver.findElement(By.id("email")).sendKeys("joseph@gmail.com");
      driver.findElement(By.id("password")).sendKeys("joseph*1996");
   }
    @When("User clicks on login")
     public void user_clicks_on_login() {
         driver.findElement(By.id("login")).click();       }
```

```
    @Then("user is navigated to the home page")
    public void user_is_navigated_to_the_home_page() throws Exception {
        driver.findElement(By.id("submitbutton")).isDisplayed();
        Boolean isLogoutDisplayed = driver.findElement(By.id("logout")).isDisplayed();
        if (isLogoutDisplayed) {
            System.out.println("Login successful and user is on the home page");
        } else {
            System.out.println("Login failed or not navigated to the home page");
        }
        Thread.sleep(2000);
        driver.close();
        driver.quit();
    }
}
```

Screenshot

```
Oct 27, 2024 7:16:13 PM cucumber.api.cli.Main run
WARNING: You are using deprecated Main class. Please use io.cucumber.core.cli.Main

Scenario: Check login is succesfull with valid credentials # src/test/resources/feature/login.feature:3
Inside step-Browser is open
  Given browser is open                                    # definitions.step_defenition.browser_is_open()
  And user is on login page                                # definitions.step_defenition.user_is_on_login_page()
  When user enters username and password                   # definitions.step_defenition.user_enters_username_and_pas
  And User clicks on login                                 # definitions.step_defenition.user_clicks_on_login()
  Then user is navigated to the home page                  # definitions.step_defenition.user_is_navigated_to_the_hom
```

Test Report

Test Case 1

Project Name:Footland					
Login Test Case					
Test Case ID: Test_1			**Test Designed By: Treesa Thomas**		
Test Priority(Low/Medium/High):			**Test Designed Date: 5-11-2024**		
Module Name: Login Module			**Test Executed By : Ms Jetty Benjamin**		
Test Title : Reader Login			**Test Execution Date: 5-11-2024**		
Description: User has a valid email/password					
Pre-Condition : User has valid username and password					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/ Fail)**
1	Navigate to login page		Login form should be displayed	Login formis displayed	Pass
2	Provide valid email	Email:treesathomas44mo@gmail.com	User should be able to login	User logs in	Pass
3	Provide valid password	Password: treesa@2002			
4	Click on login button				
Post-Condition: User is validated with database and successfully logs into account. The Account session details are logged in database					

Test Case 2: User profile

Code

```
package definitions;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import io.cucumber.java.en.And;
import io.cucumber.java.en.Given;
import io.cucumber.java.en.Then;
import io.cucumber.java.en.When;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;
import java.time.Duration;
public class profileStep {
WebDriver driver = null;
@Given("browser1 is opens")
public void browser1_is_open() {
System.out.println("Inside step-Browser is open");
System.setProperty("webdriver.gecko.marionette", "C:\\Users\\bibin\\eclipse-
workspace\\sample\\src\\test\\resources\\drivers\\geckodriver.exe");
driver = new FirefoxDriver();
driver.manage().window().maximize();
}
@And("user1 is on login pages")
public void user1_is_on_login_page() throws Exception {
driver.navigate().to("http://127.0.0.1:8000/login/");
// Using WebDriverWait instead of Thread.sleep
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("email")));
}
@When("user1 enters username and passwords")
public void user1_enters_username_and_password() throws Throwable {
driver.findElement(By.id("email")).sendKeys("treesathomas44mo@gmail.com");
driver.findElement(By.id("password")).sendKeys("treesa@2002");
}
@And("user1 clicks on logins")
public void user1_clicks_on_login() {
driver.findElement(By.id("login")).click();
}
@Then("user1 clicks on image")
public void user1_clicks_on_image() {
// Wait until the image is clickable
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement image = wait.until(ExpectedConditions.elementToBeClickable(By.id("image")));
image.click();
}
@And("user1 clicks on welcome")
public void user1_clicks_on_welcome() {
driver.findElement(By.id("welcome")).click();
```

}
}

Screenshot

```
Nov 07, 2024 3:13:42 PM cucumber.api.cli.Main run
WARNING: You are using deprecated Main class. Please use io.cucumber.core.cli.Main

Scenario: Check profile is successful with valid credentials # src/test/resources/feature/profile.feature:3
Inside step-Browser is open
  Given browser1 is opens                                # definitions.profileStep.browser1_is_open()
  And user1 is on login pages                            # definitions.profileStep.user1_is_on_login_page()
  When user1 enters username and passwords               # definitions.profileStep.user1_enters_username_and_password()
  And user1 clicks on logins                             # definitions.profileStep.user1_clicks_on_login()
  Then user1 clicks on image                             # definitions.profileStep.user1_clicks_on_image()
```

Test report

Test Case 2

Project Name:Footland	
Profile Test Case	
Test Case ID: Test_2	**Test Designed By: Treesa Thomas**
Test Priority(Low/Medium/High):	**Test Designed Date: 5-11-2024**
Module Name: User profile Module	**Test Executed By : Ms Jetty Benjamin**
Test Title : User profile	**Test Execution Date: 5-11-2024**
Description: User logs in with valid credentials and accesses the profile section.	

Pre-Condition :User has a valid email and password.

Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/ Fail)**
1	Navigate to login page	N/A	Login form should be displayed	Login formis displayed	Pass
2	Provide valid email	Email:treesathomas44mo@gmail.com	User should be able to login	User logs in	Pass
3	Provide valid password	Password: treesa@2002			
4	Click on login button				
5	Click on profile image	N/A	User should see profile options	Profile options are displayed	Pass
6	Click on "Welcome " in profile options	N/A	User is redirected to profile details page	User is redirected to profile details page	Pass

Post-Condition: The user session is authenticated, and login details are saved in the database. The user accesses the profile successfully.

Test Case 3: Cart

Code

```
package definitions;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import io.cucumber.java.en.And;
import io.cucumber.java.en.Given;
import io.cucumber.java.en.Then;
import io.cucumber.java.en.When;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;
import java.time.Duration;
public class cart {
WebDriver driver = null;
@Given("browser2 is open")
public void browser2_is_open() {
System.out.println("Inside step-Browser is open");
System.setProperty("webdriver.gecko.marionette", "C:\\Users\\bibin\\eclipse-
workspace\\sample\\src\\test\\resources\\drivers\\geckodriver.exe");
driver = new FirefoxDriver();
driver.manage().window().maximize();
}
@And("user2 is on login page")
public void user2_is_on_login_page() throws Exception {
driver.navigate().to("http://127.0.0.1:8000/login/");
// Using WebDriverWait instead of Thread.sleep
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("email")));
}
@When("user2 enters username and password")
public void user2_enters_username_and_password() throws Throwable {
driver.findElement(By.id("email")).sendKeys("treesathomas44mo@gmail.com");
driver.findElement(By.id("password")).sendKeys("treesa@2002");
}
@And("user2 clicks on login")
public void user2_clicks_on_login() {
driver.findElement(By.id("login")).click();
}
// @Then("user2 clicks on image")
// public void user2_clicks_on_image() {
// // Wait until the image is clickable
// WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
// WebElement image = wait.until(ExpectedConditions.elementToBeClickable(By.id("image")));
// image.click();
// }
// @And("user2 clicks on welcome")
// public void user2_clicks_on_welcome() {
// driver.findElement(By.id("welcome")).click();
```

```
// }
@And("user2 is on cart update page")
public void user2_is_on_cart_update_page() throws Exception {
driver.navigate().to("http://127.0.0.1:8000/cart/");
// Using WebDriverWait instead of Thread.sleep
}
}
```

Screenshot

```
Nov 07, 2024 3:27:46 PM cucumber.api.cli.Main run
WARNING: You are using deprecated Main class. Please use io.cucumber.core.cli.Main

Scenario: Check profile is successful with valid credentials # src/test/resources/feature/cart.feature:3
Inside step-Browser is open
  Given browser2 is open                                # definitions.cart.browser2_is_open()
  And user2 is on login page                            # definitions.cart.user2_is_on_login_page()
  When user2 enters username and password               # definitions.cart.user2_enters_username_and_password()
  And user2 clicks on login                             # definitions.cart.user2_clicks_on_login()
  And user2 is on cart update page                      # definitions.cart.user2_is_on_cart_update_page()
```

Test report

Test Case 3

Project Name:Footland	
Cart Test Case	
Test Case ID: Test_3	**Test Designed By: Treesa Thomas**
Test Priority(Low/Medium/High):	**Test Designed Date: 7-11-2024**
Module Name:Cart Module	**Test Executed By : Ms Jetty Benjamin**
Test Title :Cart	**Test Execution Date: 7-11-2024**
Description: User logs in and accesses the cart update page successfully.	

Pre-Condition :User has a valid email and password and is logged into the application

Step	Test Step	Test Data	Expected Result	Actual Result	Status(Pass/ Fail)
1	Open browser and navigate to login page	N/A	Login form should be displayed	Login formis displayed	Pass
2	Verify user is on login page	N/A	User sees login form	User sees login form	
3	Provide valid email	Email:treesathomas44mo@gmail.com	User should be able to login	User logs in	Pass
4	Provide valid password	Password: treesa@2002			
5	Click on login button	N/A	User successfully logs in	User is logged in	Pass
6	Navigate to cart update page	N/A	User lands on cart update page	User lands on cart update page	Pass

Post-Condition: User is successfully logged in and directed to the cart update page for further interactions.

CHAPTER 6
IMPLEMENTATION

6.1INTRODUCTION

Implementation is the stage where a planned system transforms into a tangible and operational entity. Building trust and instilling confidence in users is of paramount importance for the success of the system. This is a critical phase that places a strong emphasis on user training and the creation of informative materials. The actual transition often occurs during or after user training. Implementation signifies the act of putting a new system into action after its design phase, turning a conceptual design into a functional one.

Implementation involves the process of transitioning from the old method of operation to the new one, whether it replaces the old system entirely or makes incremental changes. Ensuring that the process is executed accurately is vital to establish a system that aligns well with the organization's requirements. System implementation encompasses the following tasks:

- Meticulous planning.
- Assessment of the existing system and its constraints.
- Designing methods to facilitate the transition.

6.2 IMPLEMENTATION PROCEDURES

Software implementation involves the installation of software in its intended location and ensuringthat it functions as intended. In some organizations, an individual who is not directly involved in using the software may oversee and approve the project's development. Initially, there may be some skepticism regarding the software, and it's essential to address these concerns to prevent strong resistance.

To ensure a successful transition, several key steps are important:

- Communicating Benefits: Users need to understand why the new software is an improvement over the old one, instilling trust in its capabilities.
- User Training: Providing training to users is crucial to make them feel confident and comfortable using the application.

To evaluate the outcome, it is essential to verify that the server program is running on the server. Ifthe server is not operational, the expected outcomes will not be achieved.

6.2.1 User Training

User training is a critical component of ensuring that individuals know how to use and adapt to a new system. It plays a vital role in fostering user comfort and confidence with the system. Even when a system becomes more complex, the need for training becomes even more apparent. User training covers various aspects, including inputting information, error handling, querying databases,and utilizing tools for generating reports and performing essential tasks. It aims to empower individuals with the knowledge and skills needed to effectively interact with the system.

6.2.2 Training on the Application Software

Once the fundamental computer skills have been covered, the next step is to instruct individuals onusing a new software program. This training should provide a comprehensive understanding of thenew system, including navigation through screens, accessing help resources, error management, and resolution procedures. The training aims to equip users or groups with the knowledge and skills necessary to effectively utilize the system or its components. It's important to note that training may be tailored differently for various groups of users and individuals in different roles within the organization to cater to their specific needs and requirements.

6.2.3 System Maintenance

Maintaining a system's functionality is a complex challenge in software development. In the maintenance phase of the software life cycle, the software performs critical functions and operates smoothly. Once a system is operational, it requires ongoing care and maintenance to ensure its continued optimal performance. Software maintenance is a crucial aspect of the development process as it ensures that the system can adapt to changes in its environment. Maintenance involvesmore than just identifying and correcting errors in the code. It encompasses various tasks that contribute to the system's stability and effectiveness

6.2.4 Hosting

Hosting refers to the process of storing and serving website files on a remote server, which can be accessed by visitors over the internet. When you create a website, you need to have it hosted on a server so that it can be available for others to view. There are various types of hosting options available such as shared hosting, dedicated hosting, VPS hosting, cloud hosting, etc. The choice of hosting depends on the size of the website, its traffic volume, and the level of control and flexibility required by the owner.

Render

Render is a cloud hosting service that provides a streamlined platform for deploying web applications, APIs, static sites, and databases. It simplifies the process of hosting applications by automating tasks such as server setup, scaling, and load balancing, which allows developers to focus on building their applications rather than managing infrastructure. Render supports various languages and frameworks, including Django, and offers seamless integration with Git for continuous deployment.

Procedure for hosting a website on 000Webhost:

Step 1: Login to your render account

Step 2: Create a new project

Step 3: Create a requirements.txt file which will include all your dependencies to be installed.

Step 4: Upload the content which is to be hosted into GitHub

Step 5: Change the setting of your project to include the host also and then deploy.

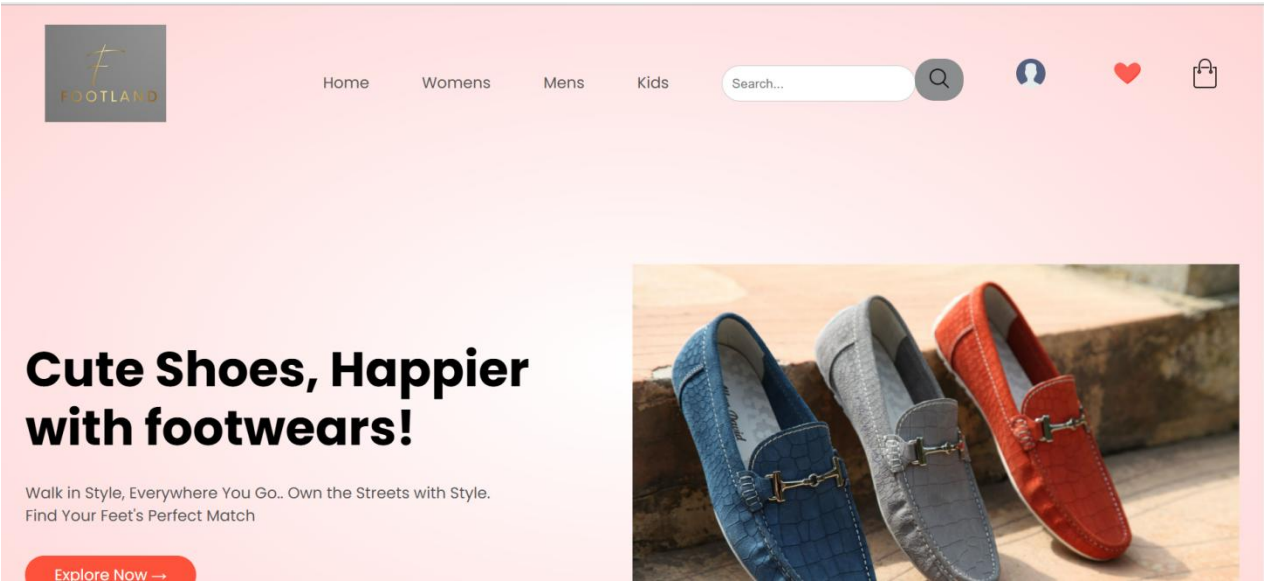
Hosted Website: Render

Hosted Link: **https://online-footwear-store-hqa7.onrender.com/**

Hosted QR code:

Footland

Screenshot


FOOTLAND
Home
Womens
Mens
Kids
Search...
Cute Shoes, Happier with footwears!
Walk in Style, Everywhere You Go.. Own the Streets with Style.
Find Your Feet's Perfect Match
Explore Now →

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

Footland is designed to provide a seamless, user-centered e-commerce experience for footwear shoppers by addressing the distinct needs of guests, registered users, and administrators. Through intuitive browsing, advanced search and filter options, and a secure checkout process, Footland streamlines online shopping for footwear. The platform enhances customer engagement by enabling account management,wish lists, and saved-for-later features, allowing registered users to personalize their shopping experience. For admins, the system offers an efficient dashboard for managing product listings, monitoring orders, and utilizing analytics tools to gain insights into sales trends and top-selling products. Additionally, future vendor support will allow third-party sellers to manage inventories and access performance analytics, enriching the catalog and adding diversity. With a responsive design, robust security features, and tailored recommendations, Footland is well-positioned to foster sales growth and customer loyalty, making it a promising platform in the online footwear market and setting a high standard for digital retail experiences.

7.2 FUTURE SCOPE

The future scope of Footland presents exciting opportunities to enhance the user experience and broaden its market impact. Key areas for future development include:

1. **Vendor Integration**: Enabling third-party vendors to manage their product listings, pricing, and stock directly on the platform.
2. **AI-Powered Recommendations**: By leveraging machine learning, Footland can deliver smarter, more personalized product recommendations tailored to each user's browsing habits, preferences, and purchase history.
3. **Enhanced Analytics for Vendors and Admins**: By introducing advanced analytics, vendors and admins could gain valuable insights into customer preferences, sales trends, and inventory needs.
4. **Loyalty and Rewards Program**: A rewards program can encourage repeat purchases by offering discounts, exclusive deals, and rewards points, building stronger customer loyalty and increasing retention.
5. **Enhanced Security Features**: By continuously updating cybersecurity measures, such as multi-factor authentication, real-time fraud detection, and secure payment processing.

6. **Social Media Integration**: Adding social media features, such as sharing user-generated content and enabling social logins, would boost Footland's visibility and drive organic traffic through social channels.
7. **Sustainable and Eco-Friendly Options**: Offering a dedicated category for eco-friendly and sustainable footwear can appeal to environmentally conscious customers, positioning Footland as a brand that values responsible consumer choices.

.

CHAPTER 8
BIBLIOGRAPHY

REFERENCES:

- "System Analysis and Design" by Gary B. Shelly and Harry J. Rosenblatt, 2009.
- "The Pragmatic Programmer: From Journeyman to Master" by Andrew Hunt and David Thomas, Pearson India, 1st Edition (2008).
- "Agile Software Development with Scrum" by Ken Schwaber and Mike Beedle, Pearson (2008).
- "Agile Testing: A Practical Guide for Testers and Agile Teams" by Lisa Crispin and Janet Gregory, Addison Wesley Professional, 1st Edition (2008).

WEBSITES:

- https://www.shoetree.io/
- https://www.metroshoes.com/
- https://inc5shop.com/
- https://www.shoppersstop.com/

CHAPTER 9
APPENDIX

9.1 Sample Code

Login Page

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Login Page</title>
   <link rel="stylesheet" href="{% static 'css/login.css' %}?v=1.0">
</head>
<body>
   <div class="container">
      <div class="image-container">
         <img src="{% static 'images/men.jpg' %}" alt="Decorative Image">
      </div>
      <div class="form-container">
         <h2>Login to continue</h2>
         <form id="loginForm" method="POST" action="{% url 'login' %}">
            {% csrf_token %}
            <div class="input-group">
               <label for="email">Email</label>
               <input type="email" name="email" id="email"
placeholder="johndoe@example.com" required>
               <span class="validation-message">Please enter a valid email address.</span>
            </div>
            <div class="input-group">
               <label for="password">Password</label>
               <div class="password-container">
                  <input type="password" name="password" id="password"
placeholder="Password" required>
                  <button type="button" class="toggle-password">
                     <img src="https://img.icons8.com/ios-filled/16/000000/visible.png" alt="Show
Password">
                  </button>
               </div>
               <span class="validation-message">Password must contain numbers and special
characters.</span>
               <a href="{% url 'forgot_password'%}" class="forgot-password">Forgot
password?</a>
            </div>
            <button type="submit" class="login-button" id="login">Login</button>
            <div class="divider">
               <hr class="line"><span>or</span><hr class="line">
            </div>
            <div class="google-signin">
               <button type="button" class="google-button">
                  <img src="https://img.icons8.com/color/16/000000/google-logo.png" alt="Google
```

```
Icon">
                Continue with Google
            </button>
        </div>
        <div class="signup-link">
            Not a member? <a href="{% url 'signup' %}">Register here</a>
        </div>
        {% if messages %}
        <div class="error-message">
            {% for message in messages %}
            <p>{{ message }}</p>
            {% endfor %}
        </div>
        {% endif %}
      </form>
    </div>
  </div>
  <script>
    document.querySelector('.toggle-password').addEventListener('click', function () {
      const passwordField = document.getElementById('password');
      const type = passwordField.getAttribute('type') === 'password' ? 'text' : 'password';
      passwordField.setAttribute('type', type);
      this.innerHTML = type === 'password' ?
        '<img src="https://img.icons8.com/ios-filled/16/000000/visible.png" alt="Show
Password">' :
        '<img src="https://img.icons8.com/ios-filled/16/000000/invisible.png" alt="Hide
Password">';
    });
  </script>
</body>
</html>
```

Views function

```
from django.contrib.auth import authenticate, login as auth_login
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib import messages
from .models import user_registration
from django.contrib.auth.hashers import make_password
from django.contrib.auth.decorators import login_required
from .models import Product,Cart,Wishlist, Review,Order
from .forms import ProductForm
from django.contrib.auth.decorators import login_required
from django.contrib.auth.models import User
from django.contrib.auth.hashers import check_password
import random
from django.urls import reverse
from django.core.mail import send_mail
from .models import UserProfile
from .forms import ProfileForm
```

```
from django.http import JsonResponse
#from django.views.decorators.csrf import csrf_exempt
import json
import logging
from django.http import HttpResponseRedirect
from .forms import ReviewForm
from django.views.decorators.cache import never_cache
from django.core.mail import send_mail
from django.conf import settings
from django.db import transaction
from decimal import Decimal  # Add this import
from django.db.models import Q
from django.contrib.auth import logout
from django.template.loader import render_to_string
from django.core.mail import EmailMessage
import io
from django.http import FileResponse
from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas
from django.template.loader import render_to_string

logger = logging.getLogger(__name__)

# Dictionary to temporarily store user pins
user_pins = {}

def home(request):
    return render(request, 'home.html')

def user_dashboard(request):
    user_id = request.session.get('user_id')
    first_name = request.session.get('first_name', "Guest")
    last_name = request.session.get('last_name', "")

    # Pass the user's first name, last name, and ID to the template
    return render(request, 'user_dashboard.html', {
        'first_name': first_name,
        'last_name': last_name,
        'user_id': user_id
    })


def login(request):
    if request.method == 'POST':
        email = request.POST['email']
        password = request.POST['password']

        # Check if it's admin login (hardcoded credentials)
        if email == 'admin@gmail.com' and password == 'admin@123':
            return redirect('admin_dashboard')
```

```
        try:
            user_obj = user_registration.objects.get(email=email)

            # Use check_password to compare the input password with the stored hashed password
            if check_password(password, user_obj.password):
                # Store user information in the session
                request.session['user_id'] = user_obj.id
                request.session['username'] = user_obj.first_name  # Assuming you store full names as
first name
                return redirect('user_dashboard')
            else:
                messages.error(request, "Invalid email or password.")
                return redirect('login')

        except user_registration.DoesNotExist:
            messages.error(request, "This account doesn't exist. Please sign up first.")
            return redirect('login')

    return render(request, 'login.html')

def forgot_password(request):
    if request.method == 'POST':
        email = request.POST.get('email')
        try:
            user = user_registration.objects.get(email=email)  # Ensure this model is correct
            # Generate a random 4-digit code
            code = random.randint(1000, 9999)
            user_pins[email] = code

            # Send email with the code
            send_mail(
                'Password Reset Code',
                f'Your password reset code is {code}.',
                'admin@yourdomain.com',  # Change to your domain
                [email],
                fail_silently=False,
            )
            # Redirect to the verification page
            messages.success(request, 'A reset code has been sent to your email.')
            verify_code_url = reverse('verify_code', kwargs={'email': email})
            return redirect(verify_code_url)
        except user_registration.DoesNotExist:  # Updated model name
            messages.error(request, 'Invalid email address.')
    return render(request, 'forgot_password.html')

# Verify code view
def verify_code(request, email):
    if request.method == 'POST':
        entered_code = request.POST.get('pin')
        correct_code = user_pins.get(email)
```

```
        if correct_code and str(entered_code) == str(correct_code):
            # Redirect to reset password page
            return redirect('reset_password', email=email)
        else:
            messages.error(request, 'Invalid code. Please try again.')

    return render(request, 'verify_code.html', {'email': email})

# Reset password view
def reset_password(request, email):
    if request.method == 'POST':
        new_password1 = request.POST.get('new_password1')
        new_password2 = request.POST.get('new_password2')

        if new_password1 == new_password2:
            try:
                user = user_registration.objects.get(email=email)  # Updated model name
                user.set_password(new_password1)  # Use set_password to hash the password
                user.save()  # Save the changes to the database
                messages.success(request, 'Password has been reset successfully.')
                return redirect('login')
            except user_registration.DoesNotExist:  # Updated model name
                messages.error(request, 'Invalid user.')
        else:
            messages.error(request, 'Passwords do not match.')
    return render(request, 'reset_password.html', {'email': email})

@never_cache
def user_dashboard(request):
    if 'user_id' in request.session:
        user_id = request.session.get('user_id')
        if user_id:
            try:
                user = user_registration.objects.get(id=user_id)
                first_name = user.first_name
            except user_registration.DoesNotExist:
                first_name = 'Guest'
        else:
            first_name = 'Guest'

        return render(request, 'user_dashboard.html', {'first_name': first_name})
    else:
        return redirect('login')


def logout(request):
    request.session.flush()  # Clears all session data
    return redirect('login')

def admin_dashboard(request):
    total_products = Product.objects.count()
```

```
    total_customers = user_registration.objects.count()
    total_vendors = 0  # You'll need to implement this based on your vendor model
    total_orders = Order.objects.count() # Assuming each unique user in Cart represents an order

    context = {
        'total_products': total_products,
        'total_customers': total_customers,
        'total_vendors': total_vendors,
        'total_orders': total_orders,
    }
    return render(request, 'admin_dashboard.html', context)

def signup(request):
    if request.method == 'POST':
        first_name = request.POST.get('first_name')
        last_name = request.POST.get('last_name')
        email = request.POST.get('email')
        password = request.POST.get('password')
        confirm_password = request.POST.get('confirm_password')

        if password != confirm_password:
            messages.error(request, 'Passwords do not match!')
            return render(request, 'signup.html')

        if user_registration.objects.filter(email=email).exists():
            messages.error(request, 'Email already exists!')
            return render(request, 'signup.html')

        new_user = user_registration(
            first_name=first_name,
            last_name=last_name,
            email=email,
            password=make_password(password)
        )
        new_user.save()

        # Customized welcome email
        subject = 'Welcome to Our Footland!'
        message = f"""
        Hi {first_name},

        Welcome to our community! We are thrilled to have you as a part of our website.

        Whether you're here to shop for your favorite items or discover something new, we strive to
provide the best experience possible. Here are a few things you can do now that you're registered:

        - Browse through our exclusive products and deals.
        - Create wishlists and share them with your friends.
        - Leave reviews for products you've purchased to help others in their buying journey.

        Should you ever need assistance or have any questions, feel free to reach out to our support
```

```
team at support@footland.com. We're always here to help.

        Thank you once again for joining us, and we hope you enjoy your time with us!

        Best regards,
        The Team

        P.S. Don't forget to follow us on social media for the latest updates and offers!
        """
        from_email = settings.DEFAULT_FROM_EMAIL
        recipient_list = [email]

        try:
            send_mail(subject, message, from_email, recipient_list)
            messages.success(request, 'Registration successful! Please check your mail!!.')
        except Exception as e:
            messages.error(request, f'Registration successful, but email failed to send: {str(e)}')

        return redirect('user_dashboard')

    return render(request, 'signup.html')

def view_customers(request):
    customers = user_registration.objects.all()  # Fetch all customers
    return render(request, 'view_customers.html', {'customers': customers})


def delete_customer(request, id):
    try:
        customer = user_registration.objects.get(id=id)
        customer.delete()
        messages.success(request, 'Customer deleted successfully!')
    except user_registration.DoesNotExist:
        messages.error(request, 'Customer does not exist!')

    return redirect('view_customers')


from django.shortcuts import render, redirect
from django.contrib import messages
from .forms import ProductForm  # Make sure to import your form
# from .models import Product  # Optional if needed for further reference

def add_product(request):
    if request.method == 'POST':
        form = ProductForm(request.POST, request.FILES)
        if form.is_valid():
            try:
                form.save()
                messages.success(request, 'Product successfully added!')
                return redirect('add_product')
```

```
            except Exception as e:
                messages.error(request, f'Error adding product: {e}')
        else:
            # Log the form errors for debugging
            messages.error(request, 'Error adding product. Please check the form.')
            for field, errors in form.errors.items():
                print(f"{field}: {errors}")
    else:
        form = ProductForm()

    return render(request, 'add_product.html', {'form': form})



def view_product(request):
    products = Product.objects.all()  # Fetch all products from the database
    return render(request, 'view_product.html', {'products': products})


def delete_product(request, product_id):
    product = get_object_or_404(Product, id=product_id)
    product.delete()
    return redirect('view_product')

def edit_product(request, product_id):
    product = get_object_or_404(Product, id=product_id)

    if request.method == 'POST':
        form = ProductForm(request.POST, request.FILES, instance=product)
        if form.is_valid():
            form.save()
            return redirect('view_product')  # Redirect to the product view page after saving
    else:
        form = ProductForm(instance=product)

    return render(request, 'edit_product.html', {'form': form, 'product': product})

def vendors(request):
    # Implement vendors logic
    return render(request, 'vendors.html')

from django.db.models import Count, Sum

def analytics(request):
    # User distribution data
    customer_count = user_registration.objects.filter(user_type='customer').count()
    admin_count = user_registration.objects.filter(user_type='admin').count()

    # Orders distribution by category
    order_data = (
        Order.objects
```

```
        .values('product__category')
        .annotate(count=Count('product__category'))
    )
    order_data_dict = {item['product__category']: item['count'] for item in order_data}

    # Best-selling products data
    best_selling_products = (
        Order.objects
        .values('product__name')
        .annotate(total_quantity=Sum('quantity'))
        .order_by('-total_quantity')[:5]
    )

    context = {
        'customer_count': customer_count,
        'admin_count': admin_count,
        'order_data': json.dumps(order_data_dict),
        'best_selling_products': best_selling_products,
    }

    return render(request, 'analytics.html', context)

@login_required
def product(request):
   return render(request, 'product.html')

def womens(request):
    return render(request, 'womens.html')

def mens(request):
    return render(request, 'mens.html')

def kids(request):
    return render(request, 'kids.html')

@login_required
def products_details(request):
    return render(request, 'products-details.html')

@login_required
def products(request):
    return render(request, 'products.html')

def search(request):
    query = request.GET.get('q')  # Fetch the search query from the GET request
    if query:
        products = Product.objects.filter(name__icontains=query)  # Filter products by name
    else:
        products = Product.objects.all()  # If no query, display all products

    context = {
```

```
        'products': products,
        'query': query
    }
    return render(request, 'search_results.html', context)

from django.core.paginator import Paginator

def search(request):
    query = request.GET.get('q')
    if query:
        products = Product.objects.filter(name__icontains=query)
    else:
        products = Product.objects.all()

    paginator = Paginator(products, 10)  # Show 10 products per page
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)

    context = {
        'products': page_obj,
        'query': query
    }
    return render(request, 'search_results.html', context)

def add_profile(request):
    user_id = request.session.get('user_id')  # Get the user ID from the session

    # Check if the user ID is set in the session
    if user_id:
        try:
            user = user_registration.objects.get(id=user_id)  # Get the user instance
        except user_registration.DoesNotExist:
            return redirect('login')  # Redirect if the user does not exist

        # Retrieve or create the user profile
        profile, created = UserProfile.objects.get_or_create(user=user)

        if request.method == 'POST':
            form = ProfileForm(request.POST, instance=profile)
            if form.is_valid():
                form.save()
                return redirect('add_profile')  # Redirect to the same page after saving
        else:
            form = ProfileForm(instance=profile)

        return render(request, 'add_profile.html', {
            'form': form,
            'first_name': user.first_name,
            'last_name': user.last_name,
            'email': user.email
        })
```

```
    # If the user is not authenticated, redirect to the login page
    return redirect('login')

def change_password(request):
    user_id = request.session.get('user_id')  # Assuming you are storing user ID in session
    if request.method == 'POST':
        new_password = request.POST.get('new_password')
        confirm_password = request.POST.get('confirm_password')

        if new_password == confirm_password:
            # Hash the password before saving it
            hashed_password = make_password(new_password)
            # Update password in the database
            user_registration.objects.filter(id=user_id).update(password=hashed_password)
            messages.success(request, 'Password changed successfully!')
            return redirect('add_profile')  # Redirect to the profile page or any other page
        else:
            messages.error(request, 'Passwords do not match.')

    return render(request, 'change_password.html', {})

def product_list(request, category):
    products = Product.objects.filter(category=category)  # Fetch products based on the category
    return render(request, 'product_list.html', {'products': products, 'category': category})

#kids product
@never_cache
def kids_products(request):
    # Filter products by category 'kids'
    products = Product.objects.filter(category='kids')

    # Get filter parameters
    price = request.GET.get('price')
    colors = request.GET.getlist('color')  # Changed to getlist to handle multiple colors
    sizes = request.GET.getlist('size')    # Changed to getlist to handle multiple sizes
    brand = request.GET.get('brand')
    types = request.GET.getlist('type')    # Changed to getlist to handle multiple types

    # Apply filters
    if price:
        products = products.filter(price__lte=price)
    if colors:
        products = products.filter(color__in=colors)
    if sizes:
        products = products.filter(size__in=sizes)
    if brand:
        products = products.filter(brand__icontains=brand)
    if types:
        products = products.filter(type__in=types)
```

```
    # Get unique values for filters
    all_colors = Product.objects.filter(category='kids').values_list('color', flat=True).distinct()
    all_sizes = Product.objects.filter(category='kids').values_list('size', flat=True).distinct()
    all_types = Product.objects.filter(category='kids').values_list('type', flat=True).distinct()

    context = {
        'products': products,
        'all_colors': all_colors,
        'all_sizes': all_sizes,
        'all_types': all_types,
        'selected_colors': colors,
        'selected_sizes': sizes,
        'selected_types': types,
        'selected_price': price,
        'selected_brand': brand,
    }
    return render(request, 'kids_products.html', context)

#womens product
@never_cache
def womens_products(request):
    # Filter products by category 'kids'
    products = Product.objects.filter(category='womens')

    # Apply filters based on GET parameters
    price = request.GET.get('price')
    color = request.GET.get('color')
    size = request.GET.get('size')
    brand = request.GET.get('brand')
    type = request.GET.get('type')


    if price:
        products = products.filter(price__lte=price)  # Filter products within price range
    if color:
        products = products.filter(color=color)  # Filter products by color
    if size:
        products = products.filter(size=size)  # Filter products by size
    if brand:
        products = products.filter(brand=brand)  # Filter products by brand
    if type:
        products = products.filter(type=type)  # Filter products by brand

    context = {
        'products': products,
    }
    return render(request, 'womens_products.html', context)

#mens product
@never_cache
def mens_products(request):
```

```
    # Filter products by category 'kids'
    products = Product.objects.filter(category='mens')

    # Apply filters based on GET parameters
    price = request.GET.get('price')
    color = request.GET.get('color')
    size = request.GET.get('size')
    brand = request.GET.get('brand')
    type = request.GET.get('type')

    if price:
        products = products.filter(price__lte=price)  # Filter products within price range
    if color:
        products = products.filter(color=color)  # Filter products by color
    if size:
        products = products.filter(size=size)  # Filter products by size
    if brand:
        products = products.filter(brand=brand)  # Filter products by brand
    if type:
        products = products.filter(type=type)  # Filter products by brand

    context = {
        'products': products,
    }
    return render(request, 'mens_products.html', context)

@never_cache
def add_to_cart(request, product_id):
    if request.method == 'POST':
        size = request.POST.get('size')
        if 'user_id' in request.session:
            user_id = request.session['user_id']  # Get the logged-in user ID from session
            user = get_object_or_404(user_registration, id=user_id)
            product = get_object_or_404(Product, id=product_id)

        # Check if the product is in stock
            if product.stock_quantity == 0:
                messages.error(request, f'Sorry, {product.name} is out of stock.')
                return redirect('product_list', category=product.category)  # Redirect with category
            else:

                cart_item, created = Cart.objects.get_or_create(user=user, product=product,size=size)

                if not created:
                    if cart_item.quantity + 1 > product.stock_quantity:
                        messages.error(request, f'Sorry, only {product.stock_quantity} units of
{product.name} are available.')
                    else:
                        cart_item.quantity += 1
                        cart_item.save()
                    messages.success(request, f'{product.name} quantity updated in your cart!')
```

```
            else:
                messages.success(request, f'{product.name} has been added to your cart!')

        return redirect('cart')  # Redirect to cart
    else:
        messages.error(request, "You need to log in to add items to the cart.")
        return redirect('login')

def view_cart(request):
    if 'user_id' in request.session:
        user_id = request.session['user_id']
        user = get_object_or_404(user_registration, id=user_id)

        # Get cart items and saved items for the logged-in user
        cart_items = Cart.objects.filter(user=user, saved_for_later=False)
        saved_items = Cart.objects.filter(user=user, saved_for_later=True)

        # Calculate the subtotal
        subtotal = sum(item.product.price * item.quantity for item in cart_items)

        # Define additional charges and tax
        shipping_charge = Decimal('0') if subtotal > 0 else Decimal('0')
        platform_fee = Decimal('0')
        tax_rate = Decimal('0')
        tax = tax_rate * subtotal

        # Total calculation
        total_amount = subtotal + shipping_charge + platform_fee + tax

        # Prepare context data for the template
        context = {
            'cart_items': cart_items,
            'saved_items': saved_items,
            'subtotal': subtotal,
            'shipping': shipping_charge,
            'platform_fee': platform_fee,
            'tax': tax,
            'total': total_amount,
        }
        return render(request, 'cart.html', context)

    else:
        messages.error(request, "You need to log in to view your cart.")
        return redirect('login')


def remove_from_cart(request, product_id):
    # Get the logged-in user ID from session
    if 'user_id' in request.session:
        user_id = request.session['user_id']
        user = get_object_or_404(user_registration, id=user_id)
```

```
        # Get the cart item for the user and the specific product, then delete it
        cart_item = get_object_or_404(Cart, user=user, product__id=product_id)
        cart_item.delete()

        # Optionally, display a success message
        messages.success(request, 'Item removed from your cart.')
    else:
        messages.error(request, "You need to log in to remove items from your cart.")
        return redirect('login')

    # Redirect back to the cart page after removing the item
    return redirect('cart')

def update_cart(request, cart_item_id):
    if request.method == "POST":
        user_id = request.session['user_id']
        user = get_object_or_404(user_registration, id=user_id)
        cart_item = get_object_or_404(Cart, id=cart_item_id, user=user)
        quantity = int(request.POST.get('quantity'))

        # Check for stock availability and update if valid
        if quantity > cart_item.product.stock_quantity:
            messages.error(request, f'Sorry, only {cart_item.product.stock_quantity} units of
{cart_item.product.name} are available.')
        elif quantity <= 0:
            messages.error(request, "Quantity must be at least 1.")
        else:
            cart_item.quantity = quantity
            cart_item.save()  # Save updated quantity to the database
            messages.success(request, "Cart updated successfully!")

    # Redirect back to `view_cart` to refresh the order summary with the updated quantity
    return redirect('cart')

from django.db import transaction

def place_order(request):
    if 'user_id' in request.session:
        user_id = request.session['user_id']
        user = get_object_or_404(user_registration, id=user_id)
        cart_items = Cart.objects.filter(user=user)

        if not cart_items.exists():
            messages.error(request, "Your cart is empty.")
            return redirect('cart')

        try:
            # Using transaction to ensure atomic operations
            with transaction.atomic():
                for item in cart_items:
```

```
                product = item.product

                # Check if the product has enough stock
                if item.quantity > product.stock_quantity:
                    messages.error(request, f'Sorry, only {product.stock_quantity} units of
{product.name} are available.')
                    return redirect('cart')

                # Reduce the stock quantity
                product.stock_quantity -= item.quantity
                product.save()

            # Empty the cart after successful order placement
            cart_items.delete()

            messages.success(request, "Your order has been placed successfully!")
            return redirect('home')
        except Exception as e:
            messages.error(request, "There was an issue placing your order. Please try again.")
            return redirect('cart')
    else:
        messages.error(request, "You need to log in to place an order.")
        return redirect('login')

def billing_details(request):
    if 'user_id' in request.session:
        user_id = request.session['user_id']
        user = get_object_or_404(user_registration, id=user_id)
        user_profile, created = UserProfile.objects.get_or_create(user=user)  # Get or create the
profile

        context = {
            'user_profile': user_profile
        }
        return render(request, 'billing_details.html', context)
    else:
        return redirect('login')

def save_billing_details(request):
    if 'user_id' in request.session:
        user_id = request.session['user_id']
        user = get_object_or_404(user_registration, id=user_id)
        user_profile, created = UserProfile.objects.get_or_create(user=user)

        if request.method == 'POST':
            # Update user details
            user.first_name = request.POST['first_name']
            user.last_name = request.POST['last_name']
            user.email = request.POST['email']
            user.save()
```

```
            # Update profile details
            user_profile.phone = request.POST['phone']
            user_profile.pincode = request.POST['pincode']
            user_profile.street = request.POST['street']
            user_profile.house_no = request.POST['house_no']
            user_profile.state = request.POST['state']
            user_profile.save()

            messages.success(request, "Billing details updated successfully!")
            return redirect('billing_details2')  # Replace with the URL of the next page
        else:
            return redirect('billing_details')
    else:
        return redirect('login')

def billing_details2(request):
    user_id = request.session.get('user_id')
    if not user_id:
        return redirect('login')

    user = get_object_or_404(user_registration, id=user_id)
    user_profile = user.userprofile
    cart_items = Cart.objects.filter(user=user, saved_for_later=False)
    total_price = sum(item.product.price * item.quantity for item in cart_items)

    context = {
        'user_profile': user_profile,
        'cart_items': cart_items,
        'total_price': total_price,
    }
    return render(request, 'billing_details2.html', context)

def confirm_order(request):
    if request.method == 'POST':
        payment_method = request.POST.get('payment_method')
        return render(request, 'confirm_order.html', {'payment_method': payment_method})
    return redirect('billing_details2')

def confirm_order_final(request):
    user_id = request.session.get('user_id')
    if not user_id or request.method != 'POST':
        return redirect('login')

    user = get_object_or_404(user_registration, id=user_id)
    confirm_choice = request.POST.get('confirm')
    payment_method = request.POST.get('payment_method')

    if confirm_choice == 'no':
        messages.info(request, "Order was canceled.")
        return redirect('billing_details2')
```

```
    cart_items = Cart.objects.filter(user=user, saved_for_later=False)
    total_price = sum(item.product.price * item.quantity for item in cart_items)

    # Create order entries and update stock
    orders = []
    for item in cart_items:
        order = Order.objects.create(
            user=user,
            product=item.product,
            quantity=item.quantity,
            size=item.size,
            total_price=item.product.price * item.quantity,
            payment_method=payment_method,
        )
        orders.append(order)
        item.product.reduce_stock(item.quantity)

    # Clear cart after order placement
    cart_items.delete()

    # Prepare order summary for email
    order_details = "\n".join([
        f"{order.product.name} - Quantity: {order.quantity} - Total: Rs. {order.total_price}"
        for order in orders
    ])
    email_body = (
        f"Hello {user.first_name},\n\nYour order has been confirmed.\n\nOrder Summary:\n"
        f"{order_details}\nTotal: Rs. {total_price}\n\nThank you for shopping with us!"
    )

    # Send confirmation email
    send_mail(
        'Order Confirmation - FootLand',
        email_body,
        settings.DEFAULT_FROM_EMAIL,
        [user.email],
        fail_silently=False
    )



    # Redirect based on payment method
    if payment_method == 'cash_on_delivery':
        messages.success(request, "Order confirmed! Confirmation email sent.")
        return redirect('order_summary')
    else:
        return redirect('online_payment')

def order_summary(request):
    user_id = request.session.get('user_id')
    if not user_id:
```

```
        return redirect('login')

    # Fetch the user's orders and profile
    orders = Order.objects.filter(user_id=user_id)
    user_profile = UserProfile.objects.get(user_id=user_id)

    return render(request, 'order_summary.html', {'orders': orders, 'user_profile': user_profile})

from reportlab.lib import colors
from reportlab.lib.pagesizes import letter, inch
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph, Spacer, Image
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.graphics.shapes import Drawing, Line
from reportlab.graphics.barcode.qr import QrCodeWidget
from reportlab.graphics import renderPDF
import qrcode
from django.contrib.staticfiles import finders

def download_order_summary(request):
    user_id = request.session.get('user_id')
    if not user_id:
        return redirect('login')

    orders = Order.objects.filter(user_id=user_id)
    user_profile = UserProfile.objects.get(user_id=user_id)

    buffer = io.BytesIO()
    doc = SimpleDocTemplate(buffer, pagesize=letter, topMargin=0.5*inch,
bottomMargin=0.5*inch)
    elements = []

    # Styles
    styles = getSampleStyleSheet()
    title_style = ParagraphStyle(
        'Title',
        parent=styles['Heading1'],
        fontSize=24,
        textColor=colors.HexColor('#1a237e'),
        spaceAfter=12,
        alignment=1
    )
    header_style = ParagraphStyle(
        'Header',
        parent=styles['Heading2'],
        fontSize=14,
        textColor=colors.HexColor('#303f9f'),
        spaceAfter=6
    )
    normal_style = ParagraphStyle(
        'Normal',
        parent=styles['Normal'],
```

```
    fontSize=10,
    textColor=colors.HexColor('#424242')
)

# Logo - Find the path of the logo
logo_path = finders.find('images/logo.jpg')
if logo_path:
    logo = Image(logo_path, width=1.5*inch, height=1.5*inch)
    elements.append(logo)
else:
    elements.append(Paragraph("Logo not found", normal_style))  # Fallback if logo is missing

# Title
elements.append(Paragraph("FootLand Order Summary", title_style))
elements.append(Spacer(1, 0.25*inch))

# Horizontal Line
d = Drawing(450, 1)
d.add(Line(0, 0, 450, 0, strokeColor=colors.HexColor('#3f51b5'), strokeWidth=2))
elements.append(d)
elements.append(Spacer(1, 0.25*inch))

# Customer Information
elements.append(Paragraph("Customer Information", header_style))
customer_info = [
    f"Name: {user_profile.user.first_name} {user_profile.user.last_name}",
    f"Email: {user_profile.user.email}",
    f"Phone: {user_profile.phone}",
    f"Address: {user_profile.address}, {user_profile.street}, {user_profile.house_no}",
    f"{user_profile.state}, {user_profile.pincode}"
]
for info in customer_info:
    elements.append(Paragraph(info, normal_style))
elements.append(Spacer(1, 0.25*inch))

# Order Details
elements.append(Paragraph("Order Details", header_style))
data = [["Product", "Quantity", "Size", "Price", "Total"]]
total_amount = 0
for order in orders:
    data.append([
        order.product.name,
        str(order.quantity),
        order.size,
        f"Rs. {order.product.price}",
        f"Rs. {order.total_price}"
    ])
    total_amount += order.total_price

table = Table(data, colWidths=[2.5*inch, 1*inch, 1*inch, 1*inch, 1.5*inch])
table.setStyle(TableStyle([
```

```
        ('BACKGROUND', (0, 0), (-1, 0), colors.HexColor('#3f51b5')),
        ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
        ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
        ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
        ('FONTSIZE', (0, 0), (-1, 0), 12),
        ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
        ('BACKGROUND', (0, 1), (-1, -1), colors.HexColor('#e8eaf6')),
        ('TEXTCOLOR', (0, 1), (-1, -1), colors.HexColor('#424242')),
        ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
        ('FONTNAME', (0, 1), (-1, -1), 'Helvetica'),
        ('FONTSIZE', (0, 1), (-1, -1), 10),
        ('TOPPADDING', (0, 1), (-1, -1), 6),
        ('BOTTOMPADDING', (0, 1), (-1, -1), 6),
        ('GRID', (0, 0), (-1, -1), 1, colors.HexColor('#c5cae9'))
    ]))
    elements.append(table)
    elements.append(Spacer(1, 0.25*inch))

    # Total Amount
    elements.append(Paragraph(f"Total Amount: Rs. {total_amount}", header_style))
    elements.append(Spacer(1, 0.25*inch))

    # QR Code
    qr = QrCodeWidget(f"Order Summary for {user_profile.user.email}")
    qr_drawing = Drawing(1*inch, 1*inch, transform=[1*inch/qr.barWidth, 0, 0,
1*inch/qr.barHeight, 0, 0])
    qr_drawing.add(qr)
    elements.append(qr_drawing)

    # Footer
    footer_style = ParagraphStyle(
        'Footer',
        parent=styles['Normal'],
        fontSize=8,
        textColor=colors.HexColor('#757575'),
        alignment=1
    )
    elements.append(Spacer(1, 0.5*inch))
    elements.append(Paragraph("Thank you for shopping with FootLand!", footer_style))

    # Build PDF
    doc.build(elements)
    buffer.seek(0)
    return FileResponse(buffer, as_attachment=True, filename="FootLand_OrderSummary.pdf")

def email_order_summary(user_profile, orders):
    # Subject and HTML content for the email
    subject = "Your Order Summary - FootLand"
    message = render_to_string('order_summary_email.html', {'user_profile': user_profile, 'orders':
orders})
```

```
    # Generate PDF attachment
    buffer = io.BytesIO()
    doc = SimpleDocTemplate(buffer, pagesize=letter, topMargin=0.5*inch,
bottomMargin=0.5*inch)
    elements = []

    # Styles
    styles = getSampleStyleSheet()
    title_style = ParagraphStyle('Title', parent=styles['Heading1'], fontSize=24,
textColor=colors.HexColor('#1a237e'), alignment=1)
    header_style = ParagraphStyle('Header', parent=styles['Heading2'], fontSize=14,
textColor=colors.HexColor('#303f9f'))
    normal_style = ParagraphStyle('Normal', parent=styles['Normal'], fontSize=10,
textColor=colors.HexColor('#424242'))

    # Add logo, title, customer info, and order details similar to `download_order_summary`
    elements.append(Paragraph("FootLand Order Summary", title_style))
    elements.append(Spacer(1, 0.25*inch))

    # Customer Information
    elements.append(Paragraph("Customer Information", header_style))
    customer_info = [
        f"Name: {user_profile.user.first_name} {user_profile.user.last_name}",
        f"Email: {user_profile.user.email}",
        f"Phone: {user_profile.phone}",
        f"Address: {user_profile.address}, {user_profile.street}, {user_profile.house_no}",
        f"{user_profile.state}, {user_profile.pincode}"
    ]
    for info in customer_info:
        elements.append(Paragraph(info, normal_style))
    elements.append(Spacer(1, 0.25*inch))

    # Order Details
    elements.append(Paragraph("Order Details", header_style))
    data = [["Product", "Quantity", "Size", "Price", "Total"]]
    total_amount = 0
    for order in orders:
        data.append([
            order.product.name,
            str(order.quantity),
            order.size,
            f"Rs. {order.product.price}",
            f"Rs. {order.total_price}"
        ])
        total_amount += order.total_price

    table = Table(data, colWidths=[2.5*inch, 1*inch, 1*inch, 1*inch, 1.5*inch])
    table.setStyle(TableStyle([
        ('BACKGROUND', (0, 0), (-1, 0), colors.HexColor('#3f51b5')),
        ('TEXTCOLOR', (0, 0), (-1, 0), colors.whitesmoke),
        ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
```

```
        ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
        ('FONTSIZE', (0, 0), (-1, 0), 12),
        ('BOTTOMPADDING', (0, 0), (-1, 0), 12),
        ('BACKGROUND', (0, 1), (-1, -1), colors.HexColor('#e8eaf6')),
        ('TEXTCOLOR', (0, 1), (-1, -1), colors.HexColor('#424242')),
        ('GRID', (0, 0), (-1, -1), 1, colors.HexColor('#c5cae9'))
    ]))
    elements.append(table)
    elements.append(Spacer(1, 0.25*inch))

    # Total Amount
    elements.append(Paragraph(f"Total Amount: Rs. {total_amount}", header_style))
    elements.append(Spacer(1, 0.25*inch))

    # Footer
    footer_style = ParagraphStyle('Footer', parent=styles['Normal'], fontSize=8,
textColor=colors.HexColor('#757575'), alignment=1)
    elements.append(Spacer(1, 0.5*inch))
    elements.append(Paragraph("Thank you for shopping with FootLand!", footer_style))

    # Build PDF
    doc.build(elements)
    buffer.seek(0)

    # Create and send email with PDF attachment
    email = EmailMessage(subject, message, to=[user_profile.user.email])
    email.attach("FootLand_OrderSummary.pdf", buffer.getvalue(), "application/pdf")
    email.send()


def online_payment(request):
    # Your logic for online payment
    return render(request, 'online_payment.html')



def add_to_wishlist(request):
    if 'user_id' not in request.session:
        messages.error(request, "Please log in to add items to your wishlist.")
        return redirect('login')

    user_id = request.session['user_id']
    user = get_object_or_404(user_registration, id=user_id)

    if request.method == "POST":
        product_id = request.POST.get('product_id')
        product = get_object_or_404(Product, pk=product_id)
        wishlist_item, created = Wishlist.objects.get_or_create(user=user, product=product)

        if created:
            messages.success(request, f'{product.name} added to your wishlist.')
```

```
        else:
            messages.info(request, f'{product.name} is already in your wishlist.')

        return redirect('product_detail', product_id=product_id)

    return redirect('home')  # Redirect to home if not a POST request
def wishlist_view(request):
    if 'user_id' not in request.session:
        messages.error(request, "Please log in to view your wishlist.")
        return redirect('login')  # Redirect to your login page

    user_id = request.session['user_id']
    wishlist_items = Wishlist.objects.filter(user_id=user_id)

    context = {
        'wishlist_items': wishlist_items,
    }
    return render(request, 'wishlist.html', context)

def remove_from_wishlist(request, product_id):
    # Get the wishlist item and delete it
    user_id = request.session['user_id']  # Get the logged-in user ID from session
    user = get_object_or_404(user_registration, id=user_id)
    wishlist_item = get_object_or_404(Wishlist, user=user, product__id=product_id)
    wishlist_item.delete()
    return redirect('wishlist')  # Redirect to the wishlist page

def product_detail(request, product_id):
    # Fetch the product details
    logger.debug(f"Fetching product with ID: {product_id}")

    product = get_object_or_404(Product, id=product_id)

    # Fetch similar products based on type
    similar_products = Product.objects.filter(
        type=product.type  # Filter by type
    ).exclude(id=product_id)[:5]  # Exclude the current product and limit to 5

    if request.method == 'POST':
        # Handle the adding to cart logic
        user = request.user  # Get the logged-in user
        if user.is_authenticated:  # Check if the user is authenticated
            quantity = request.POST.get('quantity', 1)  # Get the quantity from the form
            cart_item, created = Cart.objects.get_or_create(user=user, product=product)
            cart_item.quantity += int(quantity)  # Increase the quantity
            cart_item.save()  # Save the cart item
            return redirect('cart')  # Redirect to the cart page

    return render(request, 'product_detail.html', {
        'product': product,
        'similar_products': similar_products  # Pass similar products to the template
```

```
    })

def product_detail(request, product_id):
    product = get_object_or_404(Product, id=product_id)
    similar_products = Product.objects.filter(type=product.type).exclude(id=product_id)[:4]

    if request.method == 'POST':
        if 'add_to_cart' in request.POST:  # Handle Add to Cart form
            size = request.POST.get('size')
            quantity = int(request.POST.get('quantity', 1))
            user = request.user  # Get the logged-in user

            if user.is_authenticated:
                cart_item, created = Cart.objects.get_or_create(user=user, product=product, size=size)
                cart_item.quantity += quantity
                cart_item.save()
                messages.success(request, 'Product added to cart successfully!')
                return redirect('cart')
            else:
                messages.error(request, 'You need to be logged in to add products to the cart.')

        elif 'submit_review' in request.POST:  # Handle Review submission form
            form = ReviewForm(request.POST)
            if form.is_valid():
                review = form.save(commit=False)
                review.product = product
                review.user = request.user
                review.save()
                messages.success(request, "Your review has been submitted successfully!")
                return redirect('product_detail', product_id=product_id)
            else:
                messages.error(request, "There was an error in your review. Please check the form.")

    else:
        form = ReviewForm()

    return render(request, 'product_detail.html', {
        'product': product,
        'form': form,
        'similar_products': similar_products
    })
#def add_review(request, product_id):
    if request.method == 'POST':
        name = request.POST.get('name')
        email = request.POST.get('email')
        rating = request.POST.get('rating')
        comment = request.POST.get('comment')

        product = get_object_or_404(Product, id=product_id)

        # Create and save the review
```

```
        Review.objects.create(
            product=product,
            name=name,
            email=email,
            rating=int(rating),
            comment=comment
        )
        return redirect('product_detail', product_id=product.id)  # Redirect to product detail page

    return redirect('product_detail', product_id=product_id)  # Handle GET request by redirecting #

#def confirm_order(request):
    if request.method == 'POST':
        user_id = request.session.get('user_id')

        # Debugging: Print user_id to check if it's correctly set
        print(f"User ID from session: {user_id}")

        if user_id is None:
            messages.error(request, "User not logged in.")
            return redirect('login')  # Redirect to login if user_id is None

        try:
            user_profile = UserProfile.objects.get(user_id=user_id)  # Fetch user profile
        except UserProfile.DoesNotExist:
            messages.error(request, "UserProfile matching query does not exist.")
            return redirect('add_profile')  # Redirect to profile creation if user_profile does not exist

        # Update the user profile with the new data
        user_registration.first_name = request.POST.get('first_name')
        user_registration.last_name = request.POST.get('last_name')
        user_registration.email = request.POST.get('email')
        user_profile.address = request.POST.get('address')
        user_profile.pincode = request.POST.get('pincode')
        user_profile.phone = request.POST.get('phone')
        user_profile.save()

        # Here you would place your order logic
        # ...

        messages.success(request, "Your order has been placed successfully!")
        return redirect('home')  # Redirect to home or any other page after order is confirmed

    return redirect('cart')  # Redirect back to the cart if the method is not POST

def save_for_later(request, product_id):
    if 'user_id' in request.session:
        user_id = request.session['user_id']
        user = get_object_or_404(user_registration, id=user_id)
        cart_item = get_object_or_404(Cart, user=user, product_id=product_id)
```

```
        cart_item.saved_for_later = True
        cart_item.save()

        messages.success(request, f"{cart_item.product.name} has been saved for later.")
        return redirect('cart')
    else:
        messages.error(request, "You need to log in to save items for later.")
        return redirect('login')

def move_to_cart(request, product_id):
    if 'user_id' in request.session:
        user_id = request.session['user_id']
        user = get_object_or_404(user_registration, id=user_id)
        cart_item = get_object_or_404(Cart, user=user, product_id=product_id)

        cart_item.saved_for_later = False
        cart_item.save()

        messages.success(request, f"{cart_item.product.name} has been moved back to your cart.")
        return redirect('cart')
    else:
        messages.error(request, "You need to log in to move items to the cart.")
        return redirect('login')

def remove_saved_item(request, product_id):
    if 'user_id' in request.session:
        user_id = request.session['user_id']
        user = get_object_or_404(user_registration, id=user_id)

        # Find the item saved for later in the Cart model
        saved_item = Cart.objects.filter(product_id=product_id, user=user,
saved_for_later=True).first()

        if saved_item:
            saved_item.delete()  # Remove the item

        messages.success(request, "Item removed from saved items.")
    else:
        messages.error(request, "You need to log in to remove saved items.")

    return redirect('cart')


def my_orders(request):
    user_id = request.session.get('user_id')  # Get the user ID from the session
    if not user_id:
        return redirect('login')  # Redirect to login if the user is not logged in

    user = user_registration.objects.get(id=user_id)  # Get the current user
    orders = Order.objects.filter(user=user).order_by('-order_date')  # Get user's orders, newest first
```

```
    return render(request, 'my_orders.html', {'orders': orders})

def view_orders(request):
    # Fetch all orders for the custom orders view
    orders = Order.objects.all()
    context = {
        'orders': orders,
    }
    return render(request, 'view_orders.html', context)
```

9.2 Screen Shots

9.2.1 Login

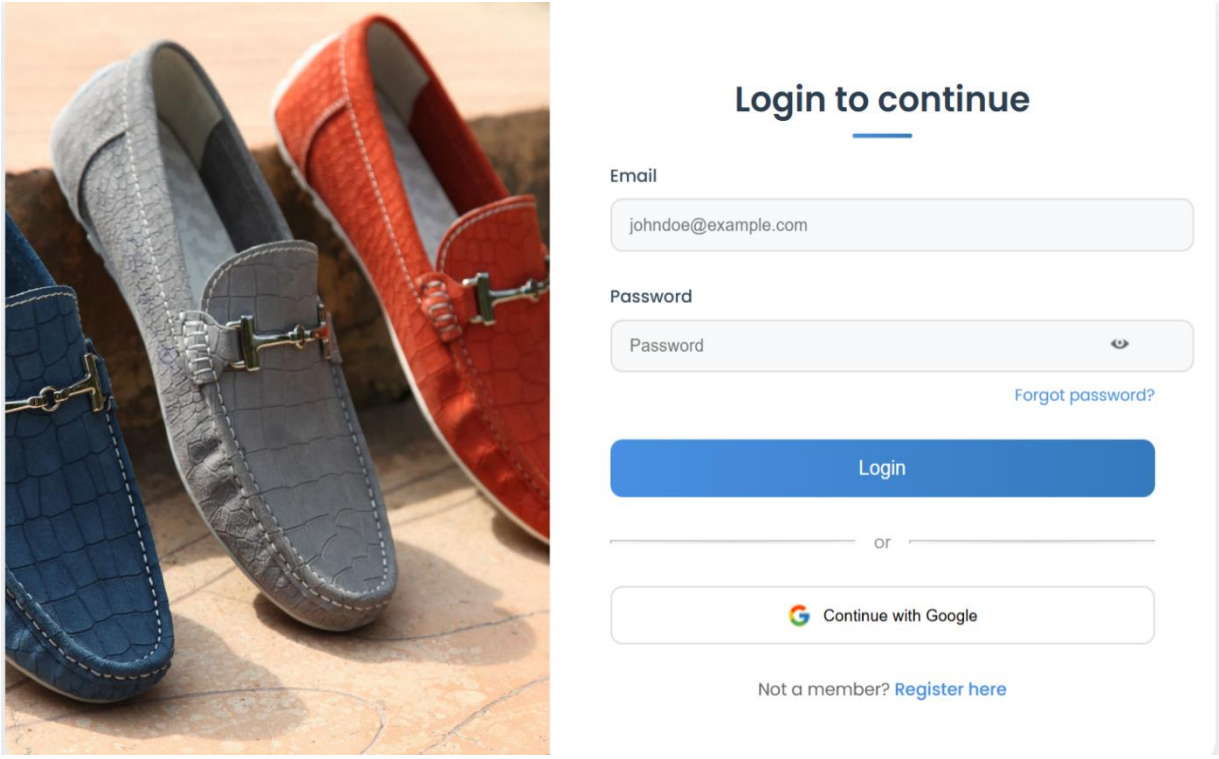


Figure 9.2.1:Login

9.2.2 Registration

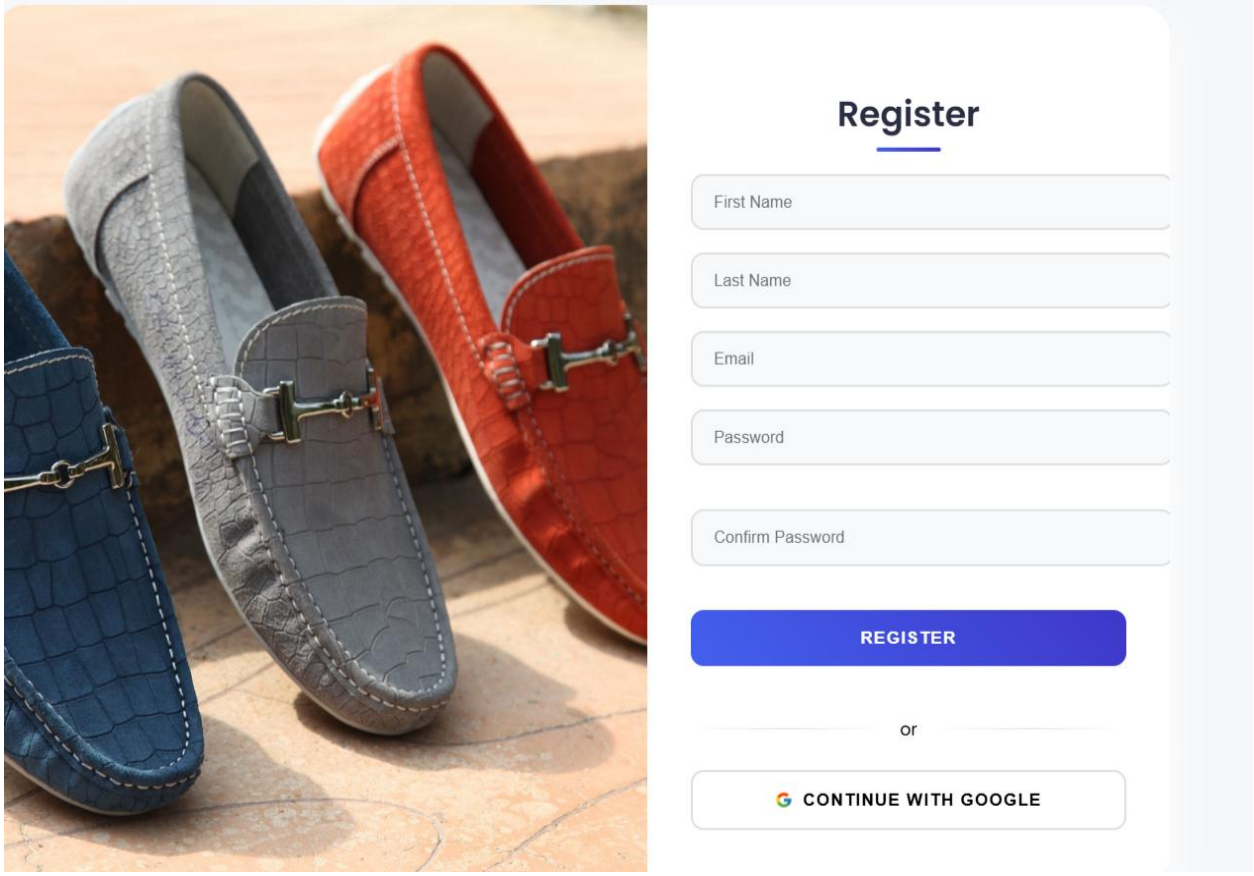


Figure:9.2.2Registration

9.2.3 Home

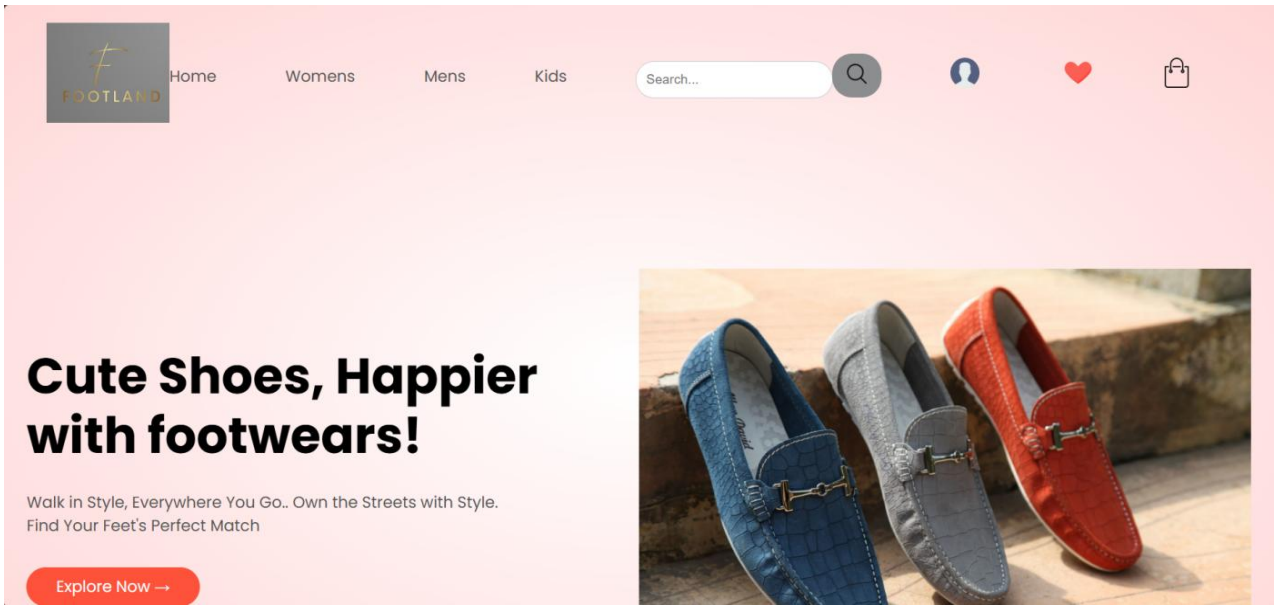


Figure:9.2.3 Home

9.2.4 User profile

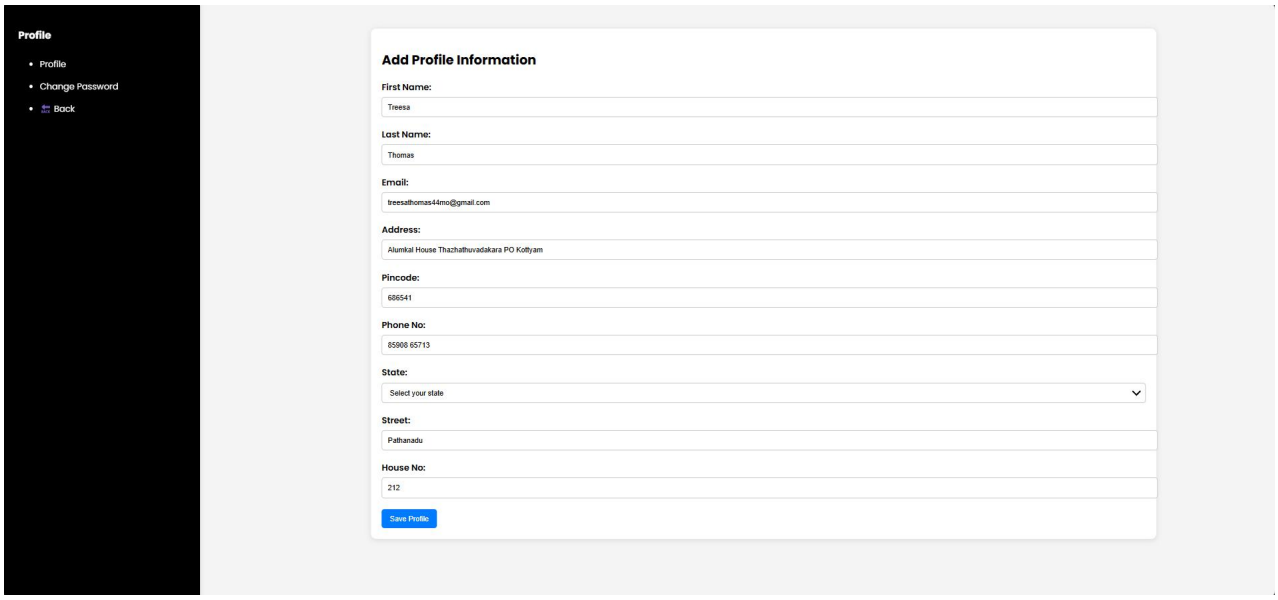


Figure:9.2.4 User profile

9.2.5 Product page

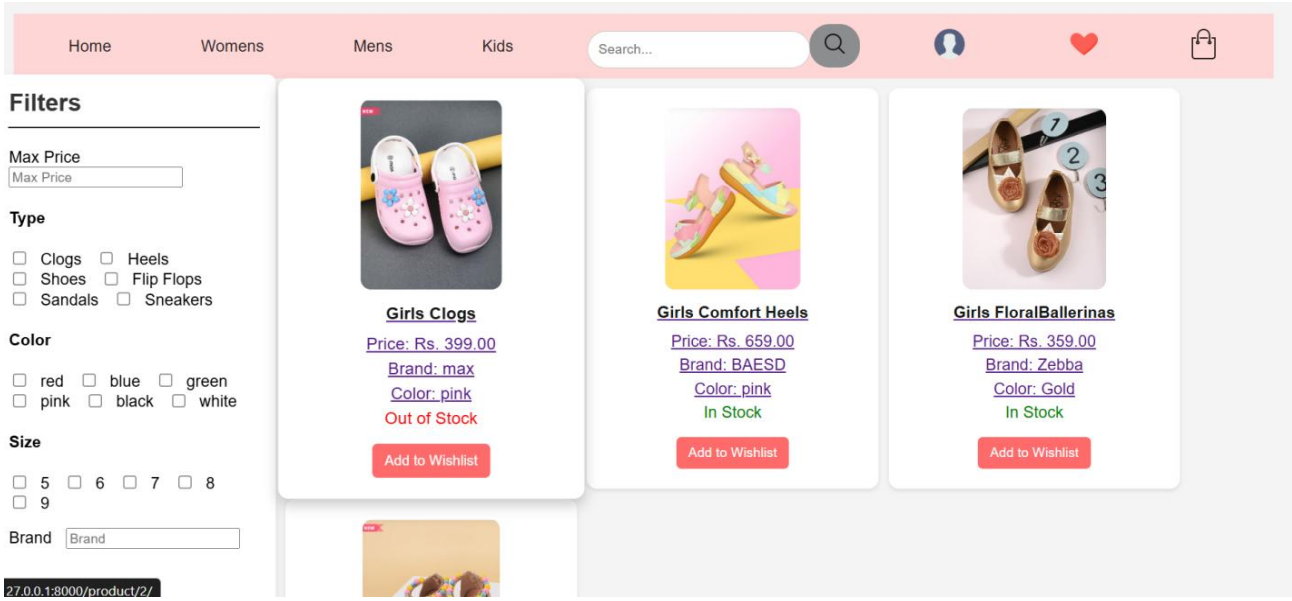


Figure:9.2.5 Product page

9.2.6 Product detail page

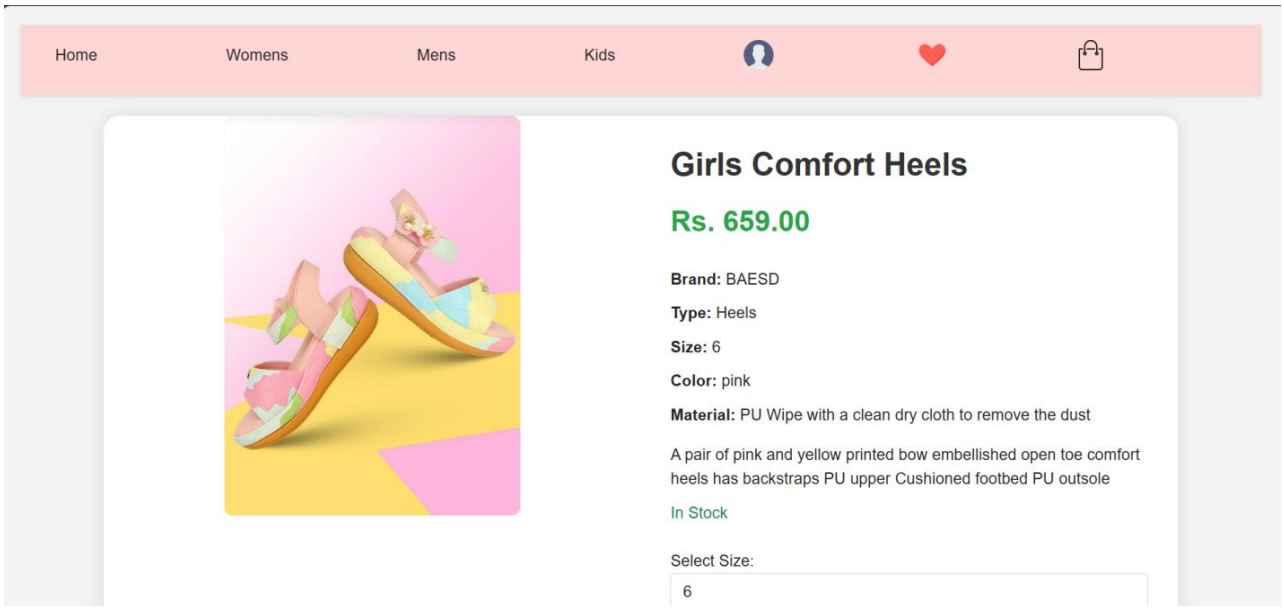


Figure:9.2.6 Product detail page

9.2.7 Wishlist page

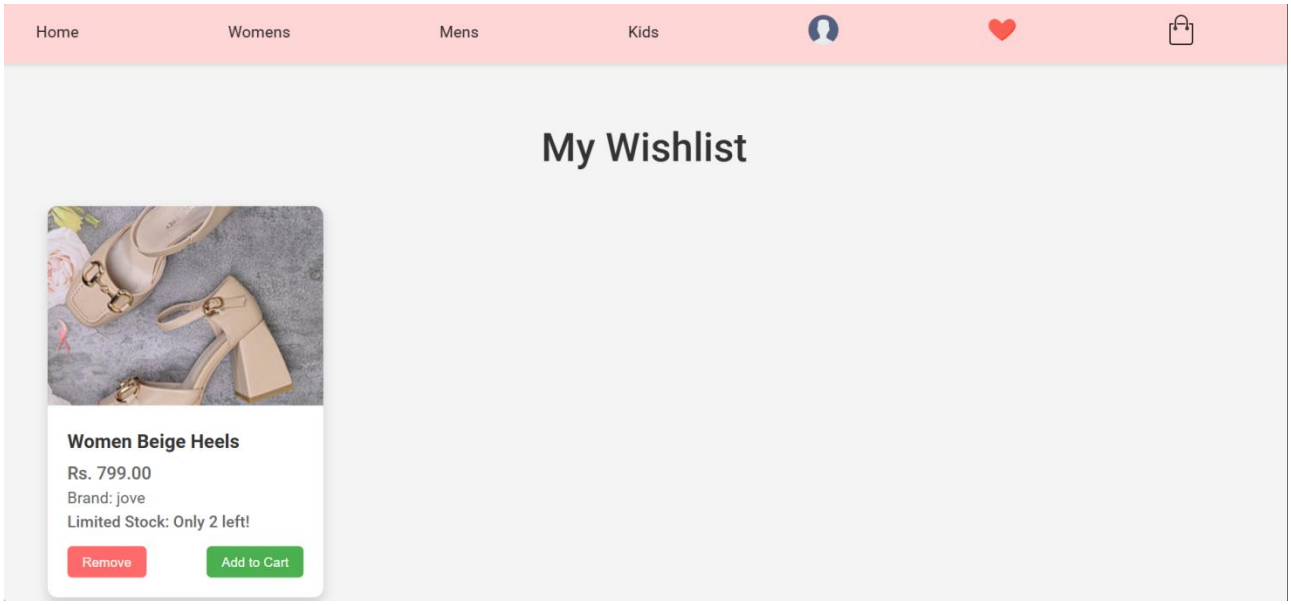


Figure:9.2.7 Wishlist page

9.2.8 Cart page

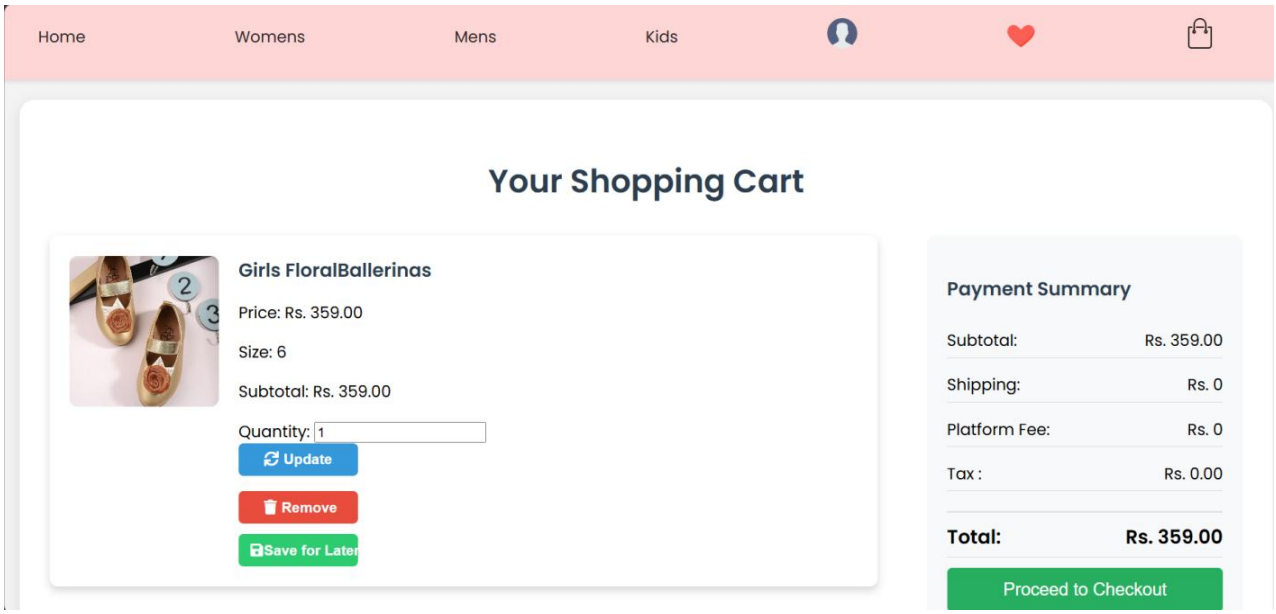


Figure:9.2.8 Cart page

9.2.9 Saved for later

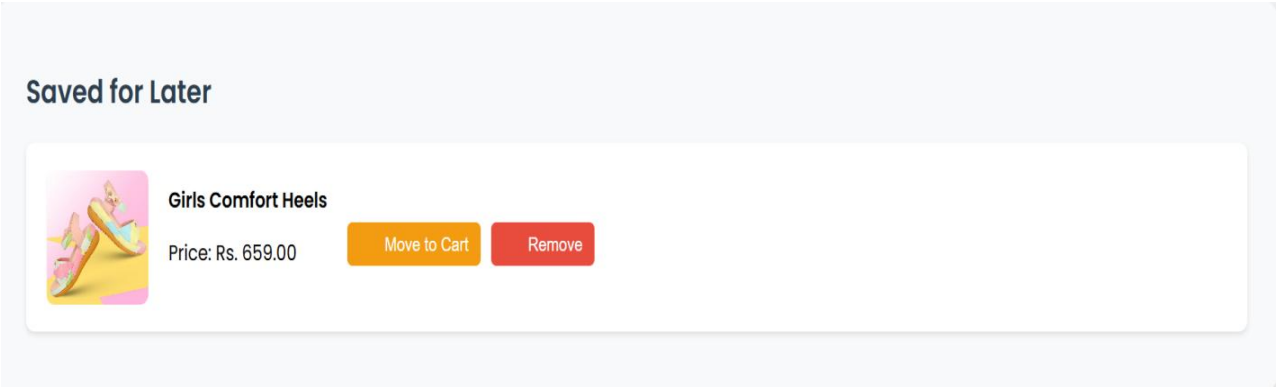


Figure:9.2.9 Saved for later

9.2.10 Admin Dashbaord

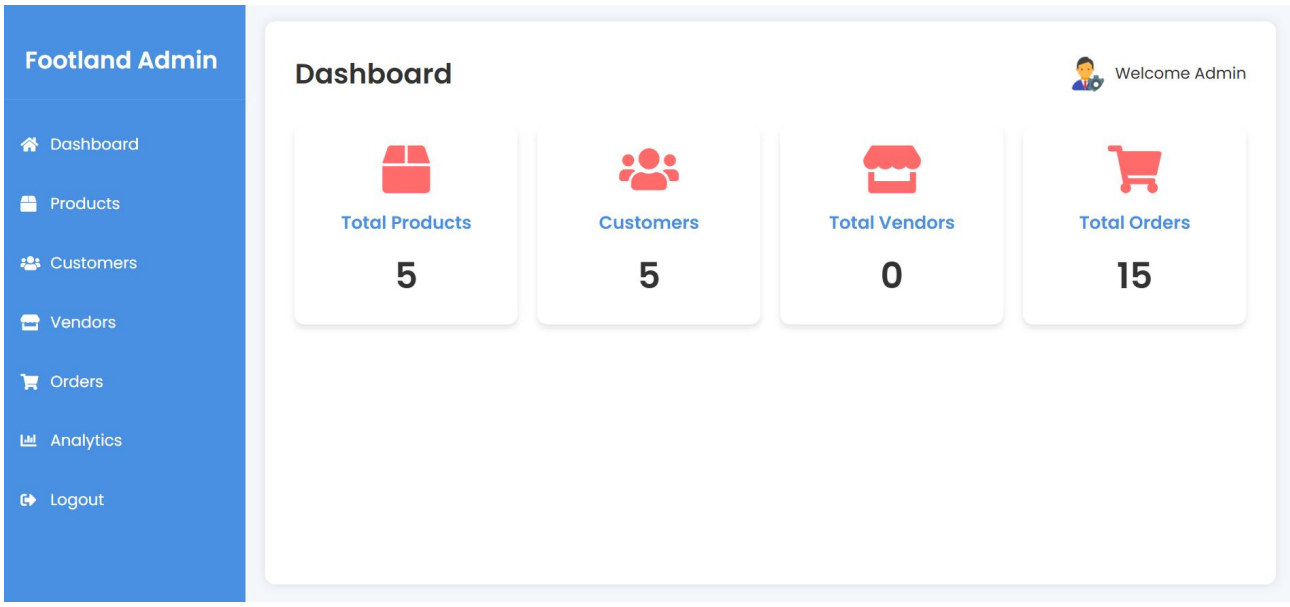


Figure 9.2.10 Admin Dashbaord

9.2.11 Product Catlouge

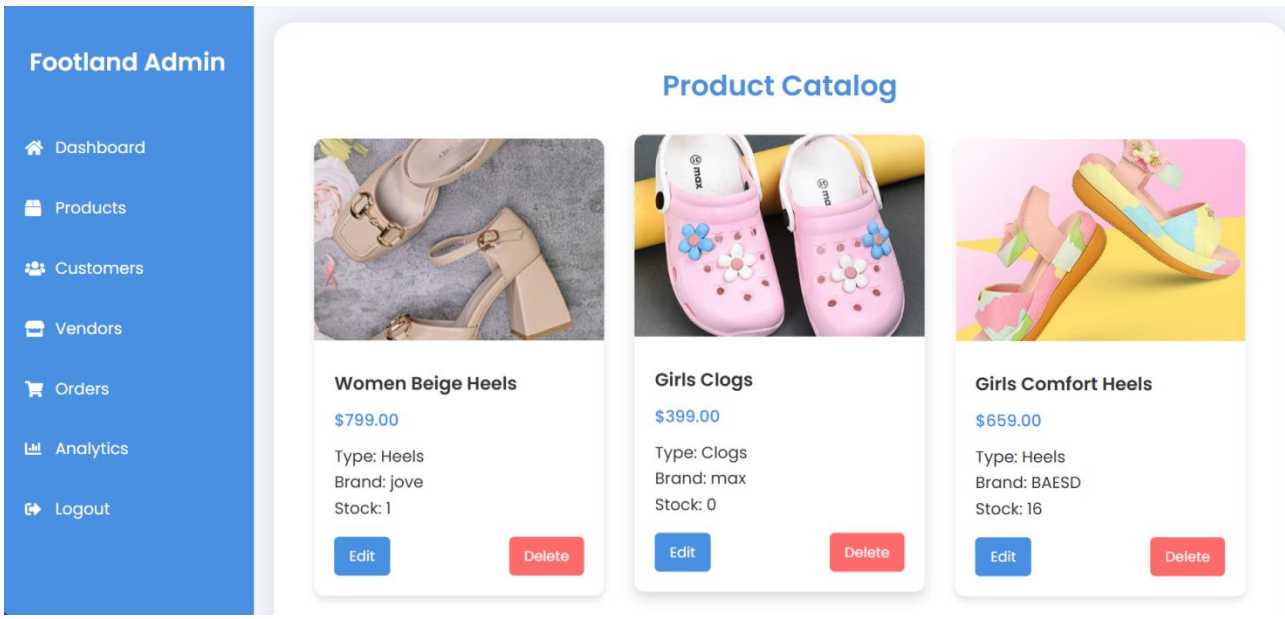


Figure 9.2.11 Product Catalog

9.2.12 Analytics Page

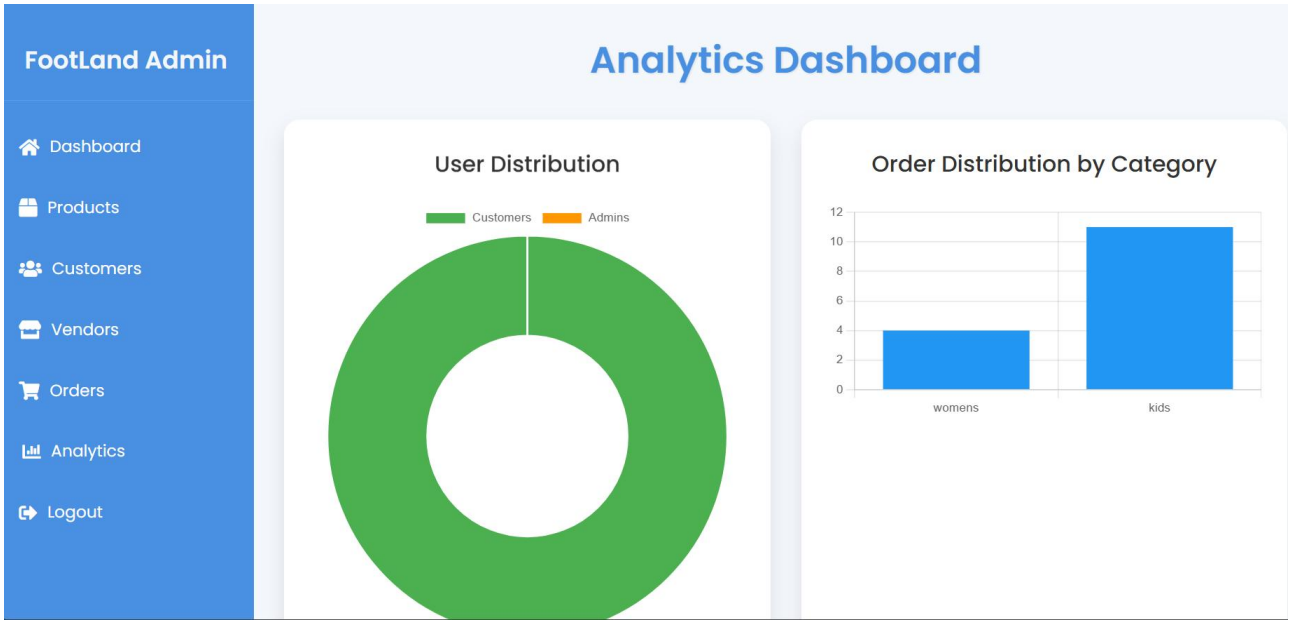


Figure 9.2.12 Analytics Page